School of Computer Science **UNIVERSITY OF LEEDS**

FACULTY OF ENGINEERING AND PHYSICAL SCIENCES

Optimizing Targeted Marketing: A Framework for Customer Segmentation and Automated Cluster-Based Notification

Dickshith Raj Nagaraj

Submitted in accordance with the requirements for the degree of

MSc Advanced Computer Science

Session 2025/2026

The candidate confirms that the following have been submitted:

Items	**Format**	**Recipient(s) and Date**
Deliverables 1–5 (this report, including the validation record and the commercial-viability assessment)	Report (PDF)	SSO / Minerva (xx/xx/xx)
Source code of the full pipeline with unit tests and reproducible outputs	Software code (Git repository / archive)	Supervisor, assessor (xx/xx/xx)
Interactive dashboard	Software (run instructions in Appendix A)	Supervisor, assessor (xx/xx/xx)

Type of Project: Exploratory Software

The candidate confirms that the work submitted is their own and the appropriate credit has been given where reference has been made to the work of others.

I understand that failure to attribute material which is obtained from another source may be considered as plagiarism.

(Signature of student) _____

Summary

This project addresses a persistent inefficiency in online retail marketing despite holding rich transactional records, retailers still deliver much of their outbound communication as undifferentiated "blanket" campaigns in which every customer receives the same message, wasting budget on customers who will not respond and fatiguing valuable customers with irrelevant contact. The problem this project set out to solve is how to convert raw retail transaction data automatically, reproducibly, and with quantified financial justification into a per customer, prioritized marketing notification plan.

The solution is an end to end software framework built in Python on the UCI Online Retail II dataset. The pipeline cleans 1,067,371 raw transaction rows down to 793,591 audited purchase lines through four sequential, logged rules, and engineers seven behavioral features (Recency, Frequency, Monetary, plus tenure, average order value, purchase cadence, and product breadth) for each of 5,878 unique customers. Six clustering algorithms spanning five algorithmic families K-Means,DBSCAN, Gaussian Mixture Models, HDBSCAN, Agglomerative (Ward), and Spectral are benchmarked under automatic hyper-parameter selection, three internal validity indices, and 50 round bootstrap stability analysis, a transparent, pre declared rule selects HDBSCAN as the operational segmentation. A predictive layer estimates each customer's 12 month lifetime value with the BG/NBD and Gamma Gamma probabilistic models and their churn probability with six supervised classifiers under a strict target leakage guard, with pairwise differences tested for statistical significance using McNemar's test. A rule based decision engine then combines segment, value tier, and churn-risk band into a concrete campaign action, channel, offer, priority, and contact time for every customer, and a 10,000-iteration Monte Carlo simulation quantifies the plan's return on investment against a static blanket-marketing baseline.

The main achievements are: (i) a fully reproducible, unit tested, version controlled pipeline that runs end to end on commodity CPU hardware in under three minutes; (ii) a methodologically rigorous segmentation in which the choice of algorithm is decided by evidence HDBSCAN ranked first overall (Silhouette 0.416) rather than by convention; (iii) an honest churn model achieving ROC-AUC 0.849, in which McNemar testing shows the two best models are statistically tied and exposes that two further models silently fail under class imbalance; and (iv) a quantified business case: the targeted plan yields a simulated mean ROI of 38.4× (95% credible interval [21.8×, 60.9×]) against 30.4× for the blanket baseline a profit uplift of +168%. The principal contribution is the closed, auditable loop from raw data to a costed action plan, a link that most published segmentation studies leave open.

Acknowledgements

I would like to thank my supervisor, Dr Fahad Panolan, for his guidance, patience, and constructive feedback throughout this project. I am also grateful to the School of Computer Science at the University of Leeds for the resources and learning environment that made this work possible, and to the maintainers of the UCI Machine Learning Repository and of the open-source Python ecosystem in particular scikit-learn, hdbscan, lifetimes, XGBoost, and Streamlit on which the implementation depends.

Table of Contents

List of Figures

List of Tables

Chapter 1
Introduction

1.1 Project Aim

The aim of this project is to design, implement, and evaluate a reproducible software framework that uses unsupervised machine learning to segment retail customers from their behavioural transaction data, and that automatically converts those segments enriched with predicted customer lifetime value and churn risk into a prioritized, per customer marketing notification plan whose financial return is quantified by simulation against an untargeted baseline.

The remainder of this section defines the concepts in that aim for a computer science reader who is new to the marketing analytics domain. **Customer segmentation** is the unsupervised partitioning of a customer base into groups whose members behave similarly; because no ground truth exists, the quality of a segmentation must be established by internal validity measures and stability analysis rather than by accuracy against labels. **RFM** Recency, Frequency, Monetary value is the standard behavioral feature framework for retail. It measures how recently, how often, and how much a customer purchases, and it remains dominant because each dimension maps directly onto an intuitive marketing meaning [15], [25]. **Clustering** algorithms discover the behavioral groups from these features. **Customer Lifetime Value (CLV)** is a forward looking estimate of the monetary value a customer will generate over a future horizon, in non contractual retail (where customers lapse silently rather than formally cancelling) it is estimated probabilistically. **Churn** is the risk that a customer stops purchasing altogether, framed here as supervised classification. Finally, a **notification plan** is the operational output the marketing team actually needs for every customer, what campaign to send, on which channel, with what offer, at what priority, and when.

The context that makes this problem worth solving is the well documented shift of Customer Relationship Management (CRM) from static profiling grouping customers by fixed demographic attributes towards dynamic, behavior driven engagement in which the next action for a customer is recomputed as their behavior changes [24], [36]. Despite that shift, a large share of outbound marketing remains untargeted. Blanket campaigns are doubly wasteful, they spend contact budget on customers who will not respond, and they risk alienating high value customers with irrelevant communication, eroding the very lifetime value the campaign was intended to protect. The practical application of this project is therefore direct it gives a small online retailer, using only its own transaction log and

commodity hardware, an auditable decision system that concentrates marketing spend where value is at risk and quantifies the expected return before any money is spent.

1.2 Objectives

The project is scoped by six objectives, each phrased so that its achievement is measurable and each discharged in an identified part of this report:

- **O1 — Background research.** Conduct a systematic literature survey of customer segmentation, clustering algorithms, customer lifetime value, churn modelling, and dynamic CRM, and use it to justify every methodological choice (Chapter 2).
- **O2 — Data foundation.** Acquire real retail transaction data and implement auditable cleaning, customer level RFM + feature engineering, and skew aware scaling (Sections 4.2–4.4).
- **O3 — Validated segmentation.** Implement and benchmark six clustering algorithms with principled, automatic hyper parameter selection; evaluate the solutions with internal validity indices and bootstrap stability; and select one segmentation by a transparent, predeclared rule (Sections 4.5–4.7 and 5.2).
- **O4 — Predictive layer.** Estimate each customer's 12-month CLV with the BG/NBD and Gamma Gamma models, and their churn probability with six classifiers compared under a target leakage guard and McNemar significance testing (Sections 4.8–4.9 and 5.3).
- **O5 — Decision layer.** Design and implement a rule based notification engine that translates segment, CLV tier, and churn band into an actionable, prioritized marketing trigger per customer, exposed through an on-demand recommendation function and an interactive dashboard (Sections 4.11 and 4.13).
- **O6 — Commercial Quantification.** Quantify the return on investment of the targeted plan, and its uplift over an untargeted static baseline, using a Monte Carlo cost benefit simulation (Sections 4.12 and 5.7).

1.3 Deliverables

The objectives map onto five concrete deliverables:

1. A version controlled Python codebase implementing the full pipeline data loading and cleaning, feature engineering, preprocessing, six algorithm clustering, cluster validation and selection, segment profiling, CLV modelling, churn classification, year on year migration analysis, the rule based notification engine, and the Monte Carlo ROI simulation orchestrated by a single entry point and configured from a single configuration module (O2–O6).

2. A reproducible experimentation and validation record: the complete set of result tables (CSV) and figures (PNG) generated by the pipeline under fixed random seeds, documenting every model selection and validation decision (O3, O4).
3. A notification logic framework mapping cluster characteristics to marketing triggers, exposed programmatically via an on-demand recommend (customer_id) function and interactively via an eight page dashboard with live customer lookup (O5).
4. A commercial viability assessment quantifying the expected financial return of the targeted plan and its uplift over static blanket marketing (O6).
5. This MSc project report (O1, and the written evaluation of all objectives).

1.4 Ethical, Legal, and Social Issues

The project raises limited but non trivial ethical, legal, social, and professional considerations, which are addressed by design rather than as an afterthought; a fuller treatment appears in Appendix B.

Legal and data protection. The dataset (UCI Online Retail II [15], [41]) is a public, secondary dataset containing no personally identifiable information beyond an anonymized numeric Customer ID: no names, addresses, contact details, or payment data are present. No attempt is made to re identify any individual, and handling follows the principles of the UK GDPR in particular purpose limitation and data minimisation, since only the fields required for behavioural modelling are read. The raw data file is treated as strictly read only; every derived artefact is written to a separate location, preserving provenance.

Ethical. The project involves no human participants and no primary data collection, so no ethical approval or consent process is required. The relevant ethical surface is instead the *use* of the system: behaviour-based targeting can, if misused, over contact customers, treat customer groups inequitably, or exploit vulnerable individuals with manipulative offers.

Social. The framework mitigates those risks structurally rather than rhetorically. The notification engine is deliberately rule based and auditable every recommendation can be traced to the exact segment, value tier, and churn band that produced it, inspected, and overridden by a human operator and contact timing is derived from each customer's own purchase rhythm rather than a fixed drumbeat, which discourages over-contact by construction. The explicit "noise" segment produced by the chosen clustering algorithm is used to *exclude* atypical customers from generic campaigns rather than force fitting them.

Professional. The work follows standard professional software practice: modular design under Git version control, a single configuration module in place of scattered constants, an automated unit test suite, fixed random seeds for reproducibility, and documentation sufficient for an independent reader to re run every stage (Appendix A). These practices

reflect the professional responsibility, articulated in the BCS Code of Conduct, to deliver reliable and maintainable analytical software whose claims can be checked.

1.5 Structure of this Report

Chapter 2 surveys the literature, reviews the candidate methods with their mathematical formulation, and justifies the chosen methods. Chapter 3 states the software requirements and presents the system design. Chapter 4 describes the implementation of every pipeline stage together with the concrete results each stage produces. Chapter 5 reports software testing and the technical and commercial evaluation. Chapter 6 concludes and outlines future work. Appendix A documents the external materials (code, artefacts, and how to run them); Appendix B expands the treatment of ethical issues.

Chapter 2
Background Research

2.1 Literature Survey

Search methodology. Candidate literature was identified by searching Scopus, IEEE Xplore, the ACM Digital Library, SpringerLink, ScienceDirect, and Google Scholar with combinations of the terms customer segmentation, RFM, clustering, K Means, DBSCAN, HDBSCAN, Gaussian mixture, customer lifetime value, BG/NBD, churn prediction, uplift modelling, and marketing ROI, over the period 2000-2026; foundational algorithmic papers were then followed backwards to their original sources regardless of date. Results were screened on title and abstract and then on full text against three inclusion criteria: (i) a peer reviewed venue or a widely cited preprint; (ii) direct relevance to at least one stage of this project's pipeline (behavioral features, clustering, validation, CLV, churn, the action layer and ROI); and (iii) either a methodological contribution or an empirical result on retail or e commerce transaction data. Comparative studies and surveys were deliberately prioritized over single method case studies, because the questions this project asks are comparative. The sources that satisfied these criteria are the ones cited below and gathered in the List of References.

2.1.1 RFM and behavioural segmentation

Customer segmentation has a long history in marketing analytics, formalized comprehensively by Wedel and Kamakura [42], and the RFM framework remains the dominant behavioral model for retail because it is interpretable and maps directly onto marketing strategy [15], [25]. The study most directly comparable to this project's data setting is Chen, Sain, and Guo [15], who applied RFM based segmentation with data mining to UK online retail data the same dataset family used here and demonstrated that behavioral segments derived from transactions are markedly more actionable than segments based on static demographic attributes, because they reflect current engagement rather than fixed identity. Later work has enriched the basic framework in two directions. Zhang, Bradlow, and Small [26] generalize RFM to RFMC by adding a "clumpiness" dimension capturing whether purchases are evenly spaced or bursty, showing that purchase regularity carries predictive value beyond recency and frequency alone; and RFM ranking or scoring schemes have been proposed as lightweight alternatives to full clustering [25]. This project follows the first direction in spirit: it retains the classical RFM core but augments it with four additional behavioral signals tenure, average order value, average inter-purchase interval (a direct, interpretable cousin of clumpiness), and product breadth chosen because each is directly meaningful to a marketing stakeholder and inexpensive to compute (Section 4.3).

RFM is not beyond criticism, and the criticisms matter for design. It is blind to *what* a customer buys (product affinity), blind to demographics, and compresses a purchase history into three summary statistics, discarding sequence information; deep sequence models can in principle exploit what RFM discards [22]. The framework nevertheless remains the right foundation for this project for three reasons. First, the downstream consumer of the segments is a marketing decision layer whose rules must be human readable (Section 3.1, NFR5), and RFM quantities are the vocabulary marketing teams already use. Second, the extended feature set recovers some of the discarded information cadence recovers timing regularity, product breadth recovers a coarse affinity signal at zero interpretability cost. Third, the probabilistic CLV models adopted in Section 2.2.5 are *defined* on RFM sufficient statistics [16], so the same feature foundation serves segmentation and value prediction coherently.

2.1.2 Comparative studies of clustering algorithms

A substantial applied literature compares clustering algorithms for segmentation, and its consistent finding motivates a central design decision of this project. Benchmarks of centroid-based methods against density based methods on retail and e commerce data repeatedly report that *no single algorithm dominates*: the appropriate choice depends on the cluster geometry of the particular dataset, the presence and desired treatment of outliers, and whether the number of clusters is known in advance [20], [21], [40]. Recent work has addressed individual weaknesses for example automating the selection of the cluster count *k* in RFM based K Means [27] but the family level trade offs remain. Segmentation studies across sectors, including banking [29] and broader machine learning driven consumer analytics [28], reach the same conclusion from different domains. The methodological implication, which this project adopts, is that the algorithm must be chosen *empirically per dataset*: six algorithms across five families are benchmarked, and the final selection is made by a pre-declared quantitative rule (Section 2.3) rather than by convention. The implication is also that *validation* not merely production of clusters is what separates rigorous segmentation from arbitrary grouping, which is why internal validity indices and bootstrap stability receive a full pipeline stage here (Sections 4.6 and 5.2).

2.1.3 Customer lifetime value

The second body of relevant work concerns probabilistic CLV estimation in non contractual settings. The canonical pairing is the BG/NBD ("Beta Geometric/Negative Binomial Distribution") model of Fader, Hardie, and Lee [17], which models the flow and cessation of a customer's repeat purchases, and the companion Gamma Gamma model of monetary value [16], which models spend per transaction under an explicit independence assumption

between transaction frequency and value. The same authors connect these models directly to RFM inputs, showing that the RFM summary is a sufficient statistic for the models likelihoods [16]. Applications span retail and financial services [30], and recent work hybridizes the probabilistic models with machine learning regressors [31]. This project adopts the classical BG/NBD + Gamma Gamma pairing (Section 4.8) because it is the established, assumption explicit approach whose diagnostics (for example the frequency value independence check) can be verified on the data.

2.1.4 Churn prediction

Churn prediction is typically framed as supervised classification over behavioral features. Ensemble tree methods random forests [8] and gradient boosted trees, most prominently XGBoost [9] generally lead on tabular retail and telecommunications benchmarks [32], and the field is comprehensively surveyed in [33]. Two methodological hazards recur across this literature and shape the design of Section 4.9. The first is target leakage: when the churn label is derived from a behavioral quantity (most commonly recency), any model given that quantity as a feature will appear nearly perfect while learning nothing useful; the honest design excludes the defining feature from the model. The second is class imbalance: churners are usually a small minority, so models optimized for accuracy or even ranking metrics can collapse to trivial behavior at the decision threshold; the standard remedies are class re weighting and threshold aware evaluation with precision recall analysis. A further, less commonly applied safeguard adopted here is formal significance testing of model differences with McNemar's test [18], which prevents over claiming a "best" model when two classifiers are statistically indistinguishable.

A final consideration the churn literature under emphasises is *error asymmetry*. In a marketing deployment the two error types have very different prices: a false positive means a retention offer (a few pounds of discount and contact cost) sent to a customer who would have stayed anyway, while a false negative means a churning customer lost silently potentially hundreds of pounds of lifetime value. A model tuned for symmetric accuracy is therefore tuned for the wrong objective; recall on the churn class deserves priority as long as precision keeps campaign costs sensible. This asymmetry, made concrete by the CLV estimates of Section 2.2.5, shapes both the class weighting choice (Eq. 17) and the evaluation emphasis on recall and precision recall analysis in Section 5.3.

2.1.5 Integrated pipelines, dynamic CRM, and uplift

Several integrated studies combine two or more of these tasks: segmentation with churn prediction on online retail data [34], hybrid RFM and clustering frameworks for churn [22], and segmentation pipelines feeding marketing strategy [35] or personalized recommendation

[37]. In parallel, the CRM literature increasingly frames analytics as an embedded component of the marketing engine, recomputing the next best action as behavior changes [24], [36]. The financial worth of targeting is, in the strongest work, expressed as *uplift* over an untargeted baseline, drawing on causal and uplift modelling techniques [38], [39] a framing adopted directly by this project's ROI simulation, which reports the *difference* between the targeted plan and a blanket baseline rather than an absolute figure alone.

2.1.6 The gap this project addresses

Across these literatures a consistent gap remains. Segmentation studies typically stop at descriptive clusters; CLV and churn studies stop at model scores; and even the integrated studies rarely close the loop to an auditable campaign action layer with quantified ROI uplift. In practical terms: the papers end where a marketing team's questions begin what should we send, to whom, when, and what will it earn us? The contribution of this project is to close that loop end to end in one reproducible system: validated clusters drive a rule based, per customer action plan whose expected financial return, and uplift over blanket marketing, is quantified by simulation before any campaign is run. The comparison below makes the positioning explicit:

Capability	**[15]**	**[34]**	**[22]**	**[35]**	**My project**
Behavioral (RFM+) segmentation	Yes	Yes	Yes	Yes	Yes
Multi-family algorithm comparison	No	Partial	No	Partial	Yes (6 algorithms, 5 families)
Stability / bootstrap validation	No	No	No	No	Yes (50 round ARI)
Probabilistic CLV	No	No	No	No	Yes (BG/NBD + Gamma–Gamma)
Churn with leakage guard + significance tests	No	Partial	Partial	No	Yes (6 models, McNemar)
Per-customer action layer (auditable)	No	No	No	Partial	Yes (rule engine + API)
ROI quantified vs untargeted baseline	No	No	No	No	Yes (Monte Carlo uplift)

Capability	[15]	[34]	[22]	[35]	My project
Reproducible end to end software	No	No	No	No	Yes (seeded, tested, dashboard)

2.2 Methods and Techniques

This section reviews the candidate methods with their mathematical formulation, so that every quantity used later in the report is defined precisely. Equations are numbered consecutively and are referred back to from Chapters 4 and 5. The notation used throughout:

Symbol	Meaning
n	number of customers (5,878 after feature engineering)
x, x_i	a customer's feature vector / feature value
μ, σ	a feature's mean and standard deviation
K	number of clusters (or classes)
C_k, μ_k	cluster k and its centroid
R_i, F_i, M_i	Recency, Frequency, Monetary of customer i
γ_1	sample skewness
$s(i)$	silhouette coefficient of point i
ARI	Adjusted Rand Index (bootstrap stability)
λ_m	m-th principal-component eigenvalue
$(r, \alpha), (a, b)$	BG/NBD purchase-rate and dropout parameters
(p, q, v)	Gamma–Gamma parameters
x, t_x, T	repeat purchases, last-purchase time, observation window
H, d	CLV horizon (12 months) and monthly discount rate (0.01)
TP, FP, FN, TN	confusion-matrix counts
b, c	discordant counts in McNemar's test
N_g, r_g, k_g	group size, response rate, conversions in simulation

2.2.1 Behavioural features and preprocessing

For customer *i* observed up to a snapshot date t_now, with purchase timestamps t_ij, per line quantities q_ij and unit prices p_ij, the three core RFM quantities are the days since the last purchase, the number of distinct invoices, and the total spend:

$$R\_i = t\_now - \max_{\top} j \ \llbracket t\_ij, \rrbracket \quad F\_i = |invoices(i)|, \quad M\_i = \Sigma\_j \, q\_ij \cdot p\_ij \qquad (1)$$

Retail spend distributions are heavy tailed, and distance-based learning is distorted when one feature's range dominates. Whether to transform a feature is decided by its sample

skewness (2); features exceeding the threshold receive the log1p transform (3) chosen over a plain logarithm because it is defined at zero, which matters for one time buyers whose tenure and inter purchase interval are exactly zero and all features are then standardised to z-scores (4):

$$\gamma\_1 = ((1/n)\ \Sigma\_i\ (x\_i - \mu)\text{^}3)/\ \sigma\text{^}3 \quad (2)$$

$$x\text{^}' = \ln(1 + x) \quad (3)$$

$$\mathrm{z} = ((\mathrm{x} - \mu))/\ \sigma \quad (4)$$

Equation (2) quantifies distributional asymmetry; (3) compresses the long right tail while preserving order; (4) places all features on a common scale with mean 0 and variance 1 so that no single feature dominates the Euclidean geometry used by the clustering algorithms.

2.2.2 Clustering algorithm families

Centroid based. K-Means [1], [2] partitions the data into K clusters $C\_1 \ldots C\_K$ with centroids μ_k by minimising the within-cluster sum of squares (the inertia):

$$\mathrm{J} = \Sigma\_\mathrm{k}\,\Sigma\_\{\mathrm{x} \in \mathrm{C\_k}\}\|\mathrm{x} - \mu\_\mathrm{k}\|\text{^}2 \quad (5)$$

It is fast and interpretable but assumes convex, similarly sized clusters and requires K in advance.

Probabilistic. A Gaussian Mixture Model [7] instead fits a weighted sum of K Gaussian components with weights π_k, means μ_k and covariances Σ_k by maximising the log-likelihood (6) via the Expectation–Maximisation algorithm, yielding soft memberships:

$$\mathrm{L} = \Sigma\_\mathrm{i}\ln\Sigma\_\mathrm{k}\,\pi\_\mathrm{k} \cdot \mathrm{N}(\mathrm{x\_i} \mid \mu\_\mathrm{k}, \Sigma\_\mathrm{k}) \quad (6)$$

Density based. DBSCAN [3], [4] defines clusters as maximal regions whose point density exceeds a threshold set by a radius ε and a minimum neighbour count, and labels low density points as noise; it does not require K but is sensitive to the single global ε. HDBSCAN [5], [6] removes that sensitivity by building the hierarchy over all density levels and keeping the clusters that are most stable across the hierarchy, retaining the noise concept without a global radius. Density methods optimise no single closed-form objective, which is why their evaluation relies entirely on the validity indices of Section 2.2.4.

Hierarchical. Agglomerative clustering merges points bottom up; Ward's linkage merges the pair of clusters whose union minimises the increase in total within cluster variance, i.e. the increase in (5), producing compact clusters comparable to K-Means but without iterative re assignment.

Graph based. Spectral clustering builds a similarity graph over the points, embeds the data using the leading eigenvectors of the graph Laplacian, and clusters in that embedded space;

it can recover non convex structure that defeats centroid methods, at higher computational cost.

Computational complexity. The families also differ in cost, which matters for the scalability discussion of Section 6.2. For n points, d features, K clusters and i iterations:

Algorithm	Time complexity	Practical note at n ≈ 5,900
K-Means	$O(n \cdot K \cdot d \cdot i)$	seconds; scales to millions of rows
GMM (EM)	$O(n \cdot K \cdot d^2 \cdot i)$	seconds at d = 7
DBSCAN	$O(n \log n)$ with spatial index	seconds; the k distance scan dominates
HDBSCAN	$O(n \log n)$ average	seconds
Agglomerative (Ward)	$O(n^2 \log n)$ time, $O(n^2)$ memory	the quadratic memory is the scaling wall
Spectral	$O(n^3)$dense; reduced via sparse k-NN affinity	tractable here only because a sparse nearest neighbor graph (10 neighbors) is used

At this project's scale every algorithm completes in seconds on CPU, so the comparison can be decided purely on quality (Section 5.2); at an order of magnitude more customers, the quadratic-memory hierarchical method and the spectral eigen decomposition would be the first casualties, which is why the framework's selection logic rather than any single algorithm is the durable artefact.

2.2.3 Hyper-parameter selection heuristics

The number of clusters for K Means is chosen by two standard heuristics reported side by side: the Elbow Method, operationalized by geometric knee detection on the inertia curve [10], and Silhouette Analysis [11], which selects the *K* maximizing the mean silhouette (7). GMM's component count is selected by minimizing the Bayesian Information Criterion, which penalizes the log likelihood (6) with a model complexity term; DBSCAN's radius ε is read from the knee of the sorted k distance plot [3]; HDBSCAN requires only a minimum cluster size; and the hierarchical and spectral methods reuse the K Means *K* to give a like for like comparison at equal cluster count. Because these heuristics can disagree and in this project they do (Section 4.5) no single heuristic is trusted alone.

2.2.4 Internal validity and stability

With no ground-truth labels, partition quality is quantified by three complementary internal indices. The silhouette coefficient (7) of point *i* contrasts its mean distance a(i) to its own cluster with its mean distance b(i) to the nearest other cluster; the Davies Bouldin index (8) averages the worst case ratio of within-cluster scatter σ to between centroid distance

$d(c\_i, c\_j)$ (lower is better) [12]; and the Calinski Harabasz index (9) is the dispersion ratio of between cluster to within-cluster scatter matrices (higher is better) [13]:

$$s(i) = ((\, b(i) - a(i)))/\ \max\{\, a(i), b(i)\}, \quad s(i) \in [-1, 1] \tag{7}$$

$$DB\ (1/K)\ \Sigma\_i\ \ \max\_\{j \neq i\}[((\sigma\_i + \sigma\_j\ ))/\ d(c\_i, c\_j\ )]\ ] \tag{8}$$

$$CH\ = [tr(B)/\ (K\ -\ 1)]/\ [tr(W)/\ (n\ -\ K)] \tag{9}$$

Geometric tightness alone does not establish that clusters are *real*: a solution can score well on indices yet change completely under small perturbations of the data. Stability is therefore measured separately by refitting each algorithm on bootstrap resamples and scoring the agreement between the full-data partition and each resampled partition with the Adjusted Rand Index (10), which corrects raw pair-counting agreement RI for chance [14]:

$$ARI\ = ((\, RI\ -\ E[RI]))/\ (\max RI - E[RI]), \quad ARI\ \in\ [-1, 1] \tag{10}$$

Finally, for visual comparison of all partitions on common axes, the scaled feature space is projected onto its first two principal components, retaining the components with the largest explained-variance ratio (11):

$$EVR\_m = \lambda\_m/\ \Sigma\_j\ \lambda\_j \tag{11}$$

2.2.5 Probabilistic customer lifetime value

The BG/NBD model [17] describes each customer by a latent purchase rate and a latent dropout process, with population level parameters (r, α) for purchasing and (a, b) for dropout. For a customer with x repeat purchases, last purchase at t_x, and observation window T, the model gives the probability the customer is still active (12) and a closed form expectation of the number of purchases in a future window of length t (13, expressible via the Gaussian hypergeometric function ${}_2F_1$):

$$P(alive \mid x, t\_x, T)\ =\ [\ 1\ +\ \delta(x > 0)\ \cdot\ (a\ /\ (b\ +\ x\ -\ 1))\ \cdot\ ((\alpha\ +\ T)/(\alpha\ +\ t\_x))\text{^}(r\ +\ x)\ ]^{-1} \tag{12}$$

$$\mathrm{E[Y(t)} \mid \mathrm{x, t\_x, T]}:\ \text{closed form in } (\mathrm{r}, \alpha, \mathrm{a}, \mathrm{b}) \text{ via the } {}_2\mathrm{F}_1 \text{ function} \tag{13}$$

The Gamma–Gamma model [16] estimates expected transaction value as a credibility weighted blend (14) of the population mean transaction value $pv/(q-1)$ and the customer's own observed mean m_x customers with few observations are shrunk towards the population mean under the assumption, checked empirically in Section 4.8, that transaction value is independent of transaction frequency. Discounting expected purchases over a horizon of H months at a monthly rate d yields the CLV (15):

$$E[M \mid m\_x, x]\ =\ ((q-1)/(px+q-1))\ \cdot\ (pv/(q-1))\ +\ (px/(px+q-1))\ \cdot\ m\_x \tag{14}$$

$$CLV\ =\ \Sigma\_\{t = 1 \ldots H\}E[purchases\_t\ ]\ \cdot E[M]/\ (1\ +\ d)\text{^}t \tag{15}$$

2.2.6 Churn labelling and classification

The churn label is defined by thresholding Recency at its 90th percentile (16); because the label is a deterministic function of Recency, Recency itself must be excluded from the model features to prevent target leakage. Class imbalance is corrected by weighting each class inversely to its frequency (17), where N is the sample size and N_c the count of class c. The baseline classifier, logistic regression, models the churn probability through the logistic function (18); tree based models (decision tree, random forest [8], gradient boosting, XGBoost [9]) instead grow splits that minimise node impurity, measured by the Gini index (19); and k-nearest neighbours classifies by distance weighted vote among the k closest customers:

$$y_i = 1 \; if \; R_i > Q_{0.90(R)}, \; else \; 0 \tag{16}$$

$$w\_c = N \; / \; (K \cdot N\_c \,) \tag{17}$$

$$P(y = 1 \mid x) \; = \; \sigma(\beta^{\mathrm{T}} x \; + \; \beta_0) \,, \quad \sigma(z) \; = \; 1/(1 \; + \; e\text{^}\{-z\}) \tag{18}$$

$$G \; = \; 1 \; - \; \Sigma_c p_c^2 \tag{19}$$

2.2.7 Evaluation metrics and significance testing

On an imbalanced task, headline metrics must be threshold aware. Precision and recall (20) measure, respectively, the purity of positive predictions and the coverage of true positives; F1 (21) is their harmonic mean; ROC AUC (22) is the probability that a randomly chosen positive is scored above a randomly chosen negative, and the area under the precision recall curve (PR AUC) summarizes threshold behavior where it matters most under imbalance:

$$Precision \; = TP/(TP + FP), \quad Recall \; = TP/(TP + FN) \tag{20}$$

$$F1 \; = \; 2 \cdot Precision \cdot Recall \; / \; (Precision \; + \; Recall) \tag{21}$$

$$ROC - AUC \; = \; P\big(\hat{s}(x^+) > \; \hat{s}(x^-) \big) \tag{22}$$

Whether two classifiers genuinely differ is tested with McNemar's procedure [18] on their discordant predictions: with b the count of test cases only the first model gets right and c the count only the second gets right, the classical statistic is (23); this project uses the exact form, evaluating $\min(b, c)$ against a $Binomial(b + c, ½)$ null, which is preferable at small discordant counts:

$$\chi\_1\text{^}2 = (|b \; - \; c| \; - \; 1)\text{^}2 / \; (b \; + \; c), \quad exact: \min(b, c) \sim Binomial(b + c, ½) \tag{23}$$

2.2.8 Monte Carlo ROI simulation

Campaign economics are uncertain chiefly through the response rate. For each campaign action group g with N_g contacted customers, the simulation draws a response rate from a

Beta prior centred on a documented planning assumption, conversions from the induced Binomial (24), and applies multiplicative Gaussian noise to per-conversion revenue; profit for one simulated world sums contribution minus contact costs (25), giving ROI (26); and the headline business quantity is the percentage profit uplift of the targeted plan over a static blanket baseline simulated by the same machinery (27):

$$r_g \sim Beta(\alpha_g, \beta_g), \quad k_g \sim Binomial(N_g, r_g) \tag{24}$$

$$Profit = \Sigma_g k_g \cdot \bar{v}_g \cdot m_g - \Sigma_g N_g \cdot cost_g, \; m_g \sim N(1, s^2) clipped > 0 \tag{25}$$

$$ROI = Profit \,/\, Cost \tag{26}$$

$$Uplift\% = 100 \cdot ((Profit\_targeted - Profit\_static))/ Profit\_static \tag{27}$$

Repeating the draw 10,000 times turns single fragile point estimates into full distributions, from which means, medians, credible intervals, and the probability of a positive return are read directly.

2.2.9 Tools and technology stack

The candidate implementation stack is Python 3.11 with pandas for data manipulation, scikit-learn [19] for clustering, classification, preprocessing, and metrics, the dedicated hdbscan library [6], the lifetimes library implementing BG/NBD and Gamma Gamma, XGBoost [9], SciPy for statistical tests, Matplotlib for figures, Streamlit for the dashboard, pytest for testing, and Git for version control. Alternatives considered include R (rich statistics but weaker application packaging for the dashboard), Spark (unnecessary at 10^6 row scale), and GPU acceleration (unnecessary at n ≈ 5,900 customers the full pipeline completes in under three minutes on CPU).

2.3 Choice of Methods

The choices are followed from the survey. Because the comparative literature (Section 2.1.2) shows that no clustering algorithm can be justified a priori, this project benchmarks six algorithms across the five families of Section 2.2.2 and selects among them with a transparent, pre declared rule: (i) disqualify any solution whose noise fraction exceeds 30%, (ii) require a mean bootstrap ARI of at least 0.70 (Eq. 10) for stability, and (iii) among the survivors choose the highest silhouette (Eq. 7). The rule encodes explicit priorities usability of the segments, then robustness, then separation and makes the selection reproducible and criticizable, which an ad hoc choice is not. HDBSCAN is included deliberately because its noise label is *marketing meaningful*: customers who fit no segment should be treated individually, not force-fitted.

The feature set extends RFM with four behavioral signals (Section 2.1.1); skewed features receive log1p (Eq. 3) before standardization (Eq. 4), with the transform decided per feature by the skew threshold $|\gamma_1| > 0.5$ (Eq. 2) rather than applied indiscriminately. CLV uses BG/NBD + Gamma Gamma (Eqs. 12–15) as the established non-contractual approach whose assumptions can be checked. Churn uses six classifiers under the leakage guard of Section 2.2.6, with class weighting (Eq. 17) where the model supports it, threshold-aware evaluation (Eqs. 20–22), and McNemar significance testing (Eq. 23) so that differences are only claimed when statistically supported. The decision layer is deliberately rule based rather than learned: auditability is a requirement (Section 3.1), and a rule table that a marketer can read, question, and override is worth more operationally than an opaque optimizer. Finally, ROI is estimated by Monte Carlo simulation (Eqs. 24–27) including a static baseline, so the reported headline is the *uplift* of targeting the quantity least sensitive to the shared planning assumptions. The overall process follows the CRISP-DM methodology [23]: business understanding (Chapter 1), data understanding and preparation (Sections 4.2–4.4), modelling (Sections 4.5–4.12), evaluation (Chapter 5), and deployment in the form of the dashboard (Section 4.13).

Chapter 3
Software Requirements and System Design

3.1 Software Requirements

This is an exploratory software project: the goal is a working, evaluated framework rather than a hardened commercial product, so the requirements are stated at the level of pipeline capabilities and system qualities. They were derived from the project aim (Section 1.1), the literature identified gap (Section 2.1.6), and the practical needs of the marketing stakeholder the system notionally serves. Each functional requirement is verified in Chapter 5 (see the traceability summary in Section 5.8).

Functional requirements.

- **FR1 — Data ingestion.** Load the raw two year transaction dataset from its source Excel workbook (both yearly worksheets) into a single typed table.
- **FR2 — Auditable cleaning.** Clean the data via documented, sequential rules and persist a cleaning audit table recording the row impact of every rule.
- **FR3 — Feature engineering.** Compute customer level RFM plus four extended behavioral features (Eq. 1; Section 4.3) against a fixed snapshot date.
- **FR4 — Preprocessing.** Apply the skew gated log1p transform and standardization (Eqs. 2–4) and persist both the scaled features and the fitted scaler.
- **FR5 — Multi-algorithm clustering.** Cluster the customers with six algorithms across five families, selecting hyper-parameters automatically (Section 2.2.3).
- **FR6 — Validation and selection.** Score every solution with the three internal indices (Eqs. 7–9) and bootstrap ARI stability (Eq. 10), rank the algorithms, and select one by the pre declared rule of Section 2.3.
- **FR7 — Profiling and naming.** Profile each selected segment in original (un scaled) units and assign deterministic marketing friendly names.
- **FR8 — CLV estimation.** Estimate per customer probability alive, expected purchases over 90/180/365-day horizons, and discounted 12 month CLV (Eqs. 12–15).
- **FR9 — Churn prediction.** Label churn by the recency percentile rule (Eq. 16), exclude the defining feature (leakage guard), train and compare six classifiers with imbalance handling (Eq. 17), and test pairwise significance (Eq. 23).
- **FR10 — Migration analysis.** Segment each trading year independently and compute the year-1 → year-2 transition matrix of segment membership.
- **FR11 — Notification plan.** Generate a per customer campaign plan (action, channel, offer, priority 1–5, contact timing) driven by segment, CLV tier, and churn band, and expose an on demand `recommend(customer_id)` lookup.

- **FR12 — ROI simulation.** Quantify the plan's ROI distribution by Monte Carlo simulation (Eqs. 24–26) including the uplift over a static blanket baseline (Eq. 27).
- **FR13 — Dashboard.** Provide an interactive dashboard over all persisted artefacts, including a live customer-lookup page.

Non-functional requirements.

- **NFR1 — Reproducibility.** Two runs of the pipeline on the same input must produce identical outputs: all stochastic steps use a fixed random seed (42) and the RFM snapshot date is pinned to the day after the last transaction rather than the wall clock.
- **NFR2 — Modularity.** One module per pipeline stage with a clear input/output contract, orchestrated centrally, so stages can be developed, tested, and re run independently.
- **NFR3 — Configurability.** Every path, threshold, and hyper parameter lives in a single configuration module; no magic numbers in stage code.
- **NFR4 — Performance and portability.** The full pipeline must run on commodity CPU hardware in minutes, with no GPU dependency (measured: ≈160 s end-to-end).
- **NFR5 — Auditability.** Every marketing recommendation must be traceable to the explicit rules and inputs that produced it, and overridable by a human.
- **NFR6 — Testability.** Core logic must be covered by automated tests that run on small synthetic datasets, independent of the raw data file.

3.2 System Design

3.2.1 Architectural overview

The system is a modular, configuration driven batch pipeline with two thin interactive consumers on top. Conceptually it forms five layers, each consuming only the outputs of the layer above: a data foundation (ingestion, cleaning, feature engineering, preprocessing), a segmentation layer (six clustering algorithms, validation, profiling), a customer intelligence layer (CLV, churn, migration), a decision layer (notification engine, ROI simulation), and a delivery layer (dashboard and artefact store). Figure 3.1 shows the layered data flow with the artefact handed between layers annotated on each arrow.

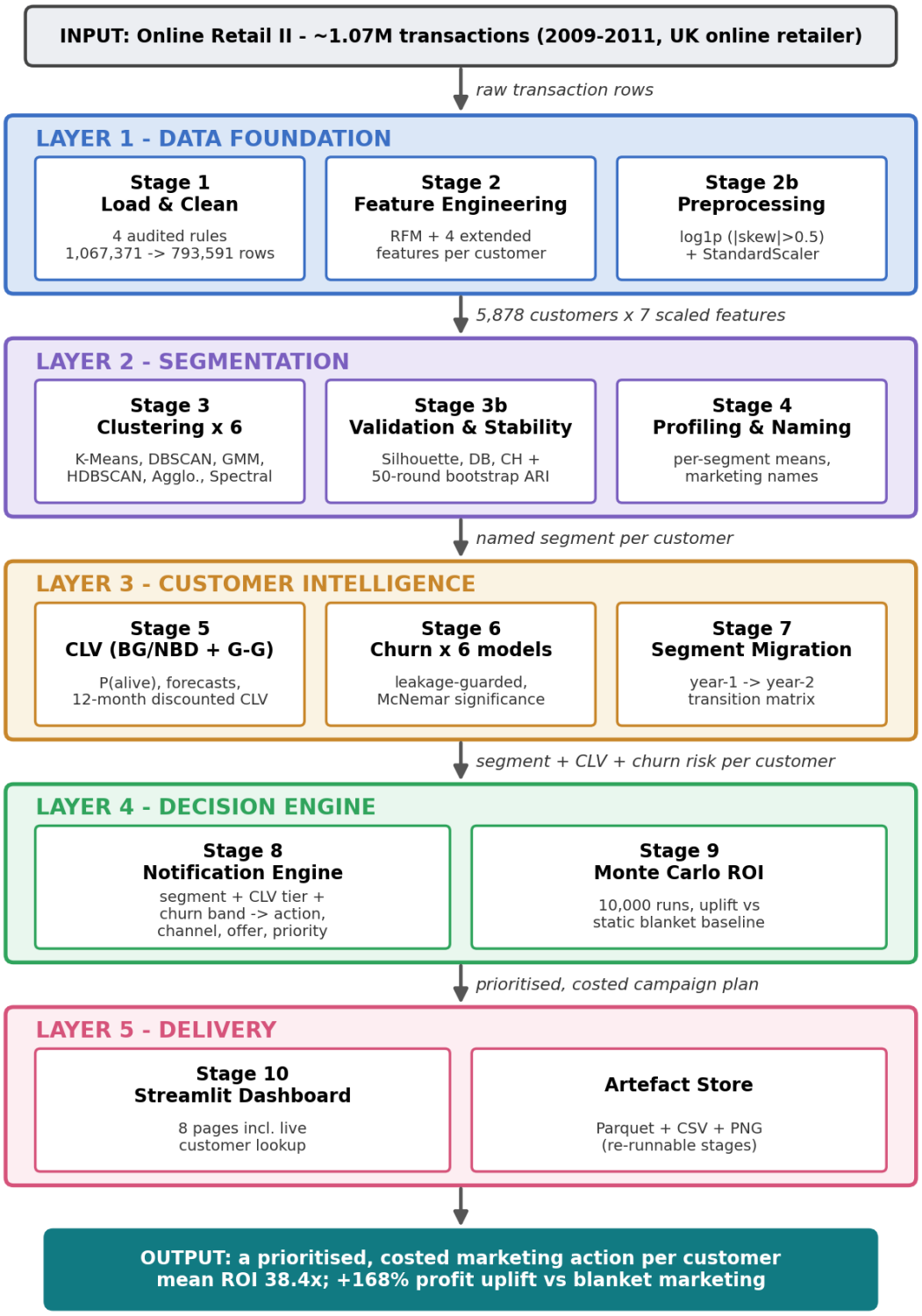

Figure 3.1 Layered system architecture and data flow of the framework, from raw transactions (top) to the costed per-customer action plan (bottom).

Data flows strictly one way — raw transactions → cleaned transactions → customer features → scaled features → cluster assignments → segment profiles → CLV and churn scores →

notification plan → ROI assessment so there are no circular dependencies between stages, and the complete state of the system after any stage is captured by the artefacts on disk. This is what makes each stage independently re runnable (NFR2) and the whole pipeline trivially debuggable: any intermediate result can be inspected as a file.

3.2.2 Component design

Figure 3.2 shows the component view. A single orchestrator (`main.py`) imports one module per stage from `src/` and executes them in dependency order. A central configuration module (`src/config.py`) is the single source of truth for every path and hyper-parameter (NFR3) stages import constants from it and never define their own. All intermediate artefacts are persisted in typed, columnar form: Parquet for datasets (efficient, schema preserving), CSV for result tables (human and Word readable), and PNG for figures. Two consumers sit on top and only *read* artefacts: the Streamlit dashboard (`app/streamlit_app.py`), and the on-demand recommendation API (`notifications.recommend(customer_id)`), which the dashboard's Customer Lookup page calls live. The automated test suite (`tests/`, pytest) exercises stage logic on synthetic data without touching the raw file.

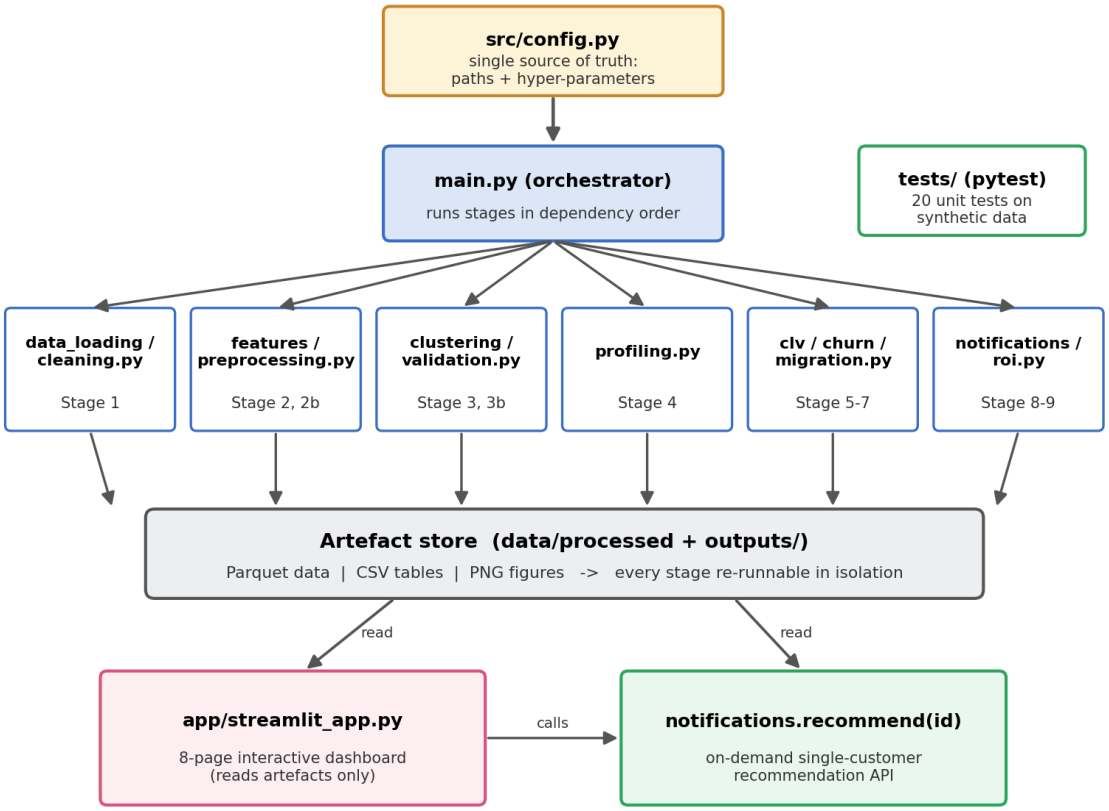

Figure 3.2 Component diagram: configuration driven orchestration, one module per stage, a persistent artefact store, and two read only consumer interfaces.

3.2.3 Stage contracts

Each stage's input/output contract is explicit, which is what makes the stages independently re runnable and testable:

Stage (module)	Consumes	Produces
1 Load + clean (data loading, cleaning)	raw workbook	cleaned_transactions.parquet; cleaning_summary.csv
2 Features (features)	cleaned transactions	customer_features.parquet; feature_summary.csv
2b Preprocessing (preprocessing)	customer features	scaled_features.parquet; fitted_scaler.joblib
3 Clustering (clustering)	scaled features	clustered_customers.parquet (6 label columns); selection figures
3b Validation (validation)	scaled features + labels	cluster_validation.csv; cluster_ranking.csv; stability figure; selected algorithm
4 Profiling (profiling)	features + selected labels	segment_profiles.csv; heatmap + radar figures
5 CLV (clv)	cleaned transactions	customer_clv.parquet; clv_summary.csv; CLV figure
6 Churn (churn)	customer features	customer_churn.parquet; comparison/ranking/McNemar tables; ROC/PR/importance figures
7 Migration (migration)	cleaned transactions	migration counts/rates tables; migration figure
8 Notifications (notifications)	clusters + CLV + churn	notification_plan.csv; recommend() API
9 ROI (roi)	notification plan + CLV	roi_simulation_summary.csv; ROI figure
10 Dashboard (app)	all persisted artefacts	interactive views (read-only)

3.2.4 Key design decisions

Four decisions shape the design. First, *artefact mediated coupling*: stages communicate only through files, never through in memory hand offs across module boundaries, which decouples development and allows the expensive analytical computation to be separated cleanly from the cheap interactive presentation. Second, *configuration centralization*: pinning every tuneable in one module makes the experiment surface explicit and the runs reproducible the configuration file effectively *is* the experiment registry. Third, *deterministic recomputation over caching cleverness*: with an end to end runtime of ≈160 s there is no need for incremental computation machinery; simplicity wins. Fourth, *rule based decision layer*: the notification engine is a readable rule table plus two modulation steps rather than a learned policy, satisfying the auditability requirement (NFR5) that Chapter 1 motivated ethically and Chapter 2 motivated operationally.

3.3 Project Management, Version Control, and Risk

The project was managed as a sequence of short, artefact driven phases: each phase corresponds to one pipeline stage, and its definition of done is that the stage's persisted artefact regenerates deterministically from a clean run. All work was version controlled in Git from the outset, with messages following the Conventional Commits style (feat(stage-3b): …, docs: …) so the history doubles as a delivery record. Table 3.1 reconstructs the milestone record directly from that history; the repository (21 development commits plus subsequent validation additions) is submitted alongside this report as the primary evidence of process.

Table 3.1 Delivery milestones as recorded in the Git commit history (messages follow the Conventional Commits convention).

Phase	Scope delivered	Commits	Completed
Scaffold & data foundation	Project structure; loading, four audited cleaning rules, RFM + extended features, preprocessing (Stages 1–2b)	4	19 Jun 2026
First clustering & validation	K-Means sweep + HDBSCAN; validity indices, bootstrap stability, selection rule (Stages 3–3b)	3	19 Jun 2026
Full algorithm benchmark	Extension to six algorithms with generalised validation (Stage 3)	1	22 Jun 2026
Value & risk models	Segment profiling and naming; BG/NBD + Gamma–Gamma CLV; six-model churn comparison (Stages 6–8)	3	22 Jun 2026
Action & commercial layer	Migration matrix; rule-based notification engine; Monte Carlo ROI with static baseline (Stages 9–11)	3	22 Jun 2026

Phase	Scope delivered	Commits	Completed
Delivery & QA	Streamlit dashboard; 20 test pytest suite; full end-to-end run; HTML/Markdown result reports (Stages 12–13)	4	22 Jun 2026
Documentation artefacts	Executed notebook generator and dissertation-grade documentation	3	23 Jun 2026
Post-draft validation	CLV temporal holdout; churn label threshold sensitivity; ROI assumption sensitivity (Sections 5.3–5.7)	1	12 Jul 2026

Risks were identified at planning time and mitigated structurally engineered into the pipeline rather than listed and forgotten. Table 3.2 records the register; every mitigation names the mechanism in the delivered system that discharges it.

Table 3.2 Project risk register: likelihood (L), impact (I), and the mitigation engineered into the delivered system.

Risk	L	I	Mitigation engineered into the system
Clustering yields no stable or usable structure on this data	Med	High	Six algorithm benchmark with a pre declared selection rule and a bootstrap ARI stability gate; K Means retained as the conventional fallback (Section 4.6)
Target leakage silently inflates the churn results	Med	High	Recency excluded from the feature set by design; the guard is enforced by a dedicated unit test (Sections 4.9, 5.1)
Probabilistic model fitting fails to converge	Med	Med	Penalized fitting with a documented parameter ladder; convergence logged; degenerate boundary fits rejected (Sections 4.8, 5.4)

Risk	**L**	**I**	**Mitigation engineered into the system**
Data quality defects corrupt every downstream stage	High	Med	Four audited cleaning rules with a persisted row-impact audit (Table 4.1); the raw workbook is treated as read-only
Results not reproducible at assessment time	Low	High	Single seed and pinned snapshot date centralized in config.py; byte-identical rerun check (Section 5.1); Git history
Scope creep beyond the assessable core	Med	Med	Phased delivery with per phase definition of done; stage contracts freeze inter module interfaces early (Section 3.2.3)

Chapter 4
Software Implementation

The system is implemented in Python 3.11, with one module per pipeline stage under `src/`, orchestrated by `main.py`. This chapter walks through the implementation of every stage in pipeline order, quantifying what each stage does to the data and showing the artefacts it produces. Equation references point back to the formulations of Section 2.2.

4.1 Environment and Project Layout

The project depends on pandas, scikit-learn [19], hdbscan [6], lifetimes, XGBoost [9], SciPy, Matplotlib, Streamlit, and pytest (full listing in Appendix A). The repository layout mirrors the architecture: `src/` holds twelve stage modules plus configuration; `app/` the dashboard; `tests/` the automated test suite; `data/raw/` the read-only source workbook; and `data/processed/` and `outputs/` the artefact store (five Parquet datasets, sixteen result tables, nineteen figures). All randomised procedures the train/test split, cross-validation folds, bootstrap resampling, and the Monte Carlo simulation draw from a fixed seed (42), and the RFM snapshot date is pinned to the day after the final transaction in the data, so every run of the pipeline reproduces every number in this report exactly (NFR1).

4.2 Data Loading and Cleaning

The two yearly worksheets of the Online Retail II workbook are loaded and concatenated into a single frame of **1,067,371** raw rows (FR1). Cleaning (FR2) then applies four sequential rules, each measured against the already-filtered data so that row impacts are attributed uniquely and do not double-count:

1. **Drop missing Customer ID** — removes 243,007 rows (22.77%). Anonymous, till-style transactions cannot be attributed to any customer and are unusable for customer-level modelling.
2. **Remove cancellations** — invoices prefixed "C" are credit notes; 18,744 rows (2.27%). Including them would double-count reversed revenue.
3. **Remove non-positive quantity or sub-penny price** — physically meaningless or non-sale lines (manual adjustments, damaged-stock write-offs); 89 rows (0.01%).
4. **Drop exact duplicates** — identical repeated rows almost certainly represent capture errors; 11,940 rows (1.48%).

After cleaning, **793,591** transaction lines remain a total reduction of 25.65% from which line revenue (quantity × price) is derived and column types are enforced. The stage's logic, in pseudocode:

```
frames = [read_sheet("Year 2009-2010"), read_sheet("Year 2010-2011")]
df = concat(frames)                                    # 1,067,371 rows
audit = []
for rule in [drop_missing_customer_id,                 # applied in this order,
             drop_cancellation_invoices,               # each measured against the
             drop_nonpositive_qty_or_price,            # already filtered frame
             drop_exact_duplicates]:
    before = len(df)
    df = rule(df)
    audit.append(rule.name, before, len(df), before - len(df), pct)
df["Revenue"] = df.Quantity * df.Price
persist(df, CLEANED_PARQUET);  persist(audit, CLEANING_SUMMARY_CSV)
```

Table 4.1 is the persisted audit table itself: making every exclusion measurable and justifiable is the point of FR2, and this table is the evidence an examiner or auditor would ask for.

Table 4.1 Data cleaning audit: the row impact of each sequential rule.

Step	Description	Rows Before	Rows After	Rows Removed	% Removed
1	Drop missing Customer ID	1,067,371	824,364	243,007	22.77
2	Remove cancelled invoices (prefix 'C')	824,364	805,620	18,744	2.27
3	Remove Quantity < 1 or Price < 0.01	805,620	805,531	89	0.01
4	Drop exact duplicate rows	805,531	793,591	11,940	1.48

4.3 Feature Engineering

From the cleaned transactions, the pipeline computes seven customer level behavioral features (FR3) for each of the **5,878** unique customers, using the fixed snapshot date (NFR1). The three RFM features follow Eq. (1), with Frequency deliberately counting *distinct invoices* rather than transaction lines a customer who buys forty items in one order has shopped once, not forty times. Four extended features sharpen the behavioral picture: **Tenure** (days between first and last purchase 0 for one time buyers), **AvgOrderValue** (Monetary/Frequency), **AvgInterPurchaseDays** (mean gap between consecutive purchases the customer's natural rhythm, reused by the notification engine in Section 4.11), and **DistinctProducts** (breadth of the product relationship).

The full feature set, with definitions and rationale:

Feature	Definition	Marketing meaning
Recency	days from last purchase to snapshot (Eq. 1)	engagement freshness the churn label basis (Eq. 16)
Frequency	count of distinct invoices (Eq. 1)	habit strength
Monetary	total spend across all lines (Eq. 1)	realised value

Feature	Definition	Marketing meaning
Tenure	days between first and last purchase	relationship length; 0 identifies one time buyers
AvgOrderValue	Monetary / Frequency	basket economics; feeds ROI revenue model (Eq. 25)
AvgInterPurchaseDays	mean gap between consecutive purchases	natural rhythm; drives contact timing (Eq. 29)
DistinctProducts	count of unique products purchased	breadth of the relationship

The computed population is heavily skewed, as expected of retail: median Monetary is £887 against a mean of £3,009 and a maximum of £608,822; median Frequency is 3 against a maximum of 398; median Recency is 96 days against a mean of 201 (the two year window spans 739 days). Table 4.2 gives the full descriptive statistics and Figure 4.1 shows the RFM distributions; the long right tails motivate the preprocessing stage that follows.

Table 4.2 Descriptive statistics of the seven engineered features across the 5,878 customers.

	count	mean	std	min	25%	50%	75%	max
Customer ID	5,878	1.532e+04	1,716	12,346	1.383e+04	1.531e+04	1.68e+04	18,287
Recency	5,878	201.3	209.3	1	26	96	380	739
Frequency	5,878	6.289	13.01	1	1	3	7	398
Monetary	5,878	3,009	1.473e+04	2.95	345.1	887.4	2,298	6.088e+05
Tenure	5,878	273	258.8	0	0	220.5	511	738
AvgOrderValue	5,878	390	1,215	2.95	179.4	283.6	419.7	8.424e+04
AvgInterPurchaseDays	5,878	50.15	54.85	0	0	36.65	76.08	357
DistinctProducts	5,878	81.99	116.5	1	19	45	103	2,550

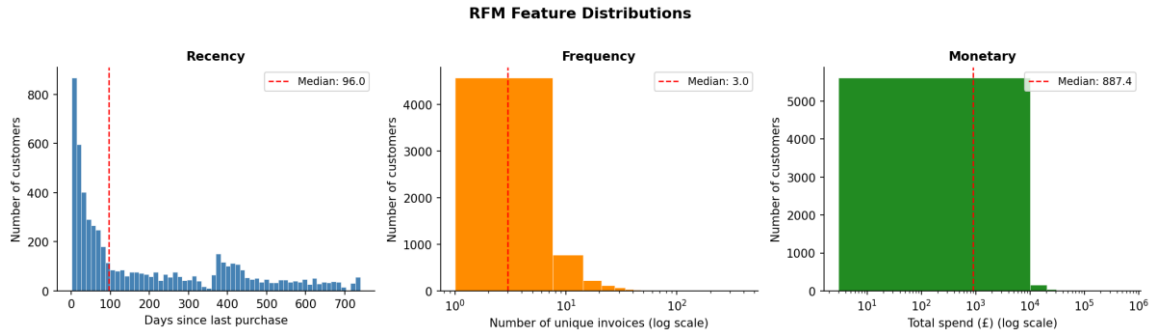

Figure 4.1 Distributions of the RFM features across the 5,878 customers (Frequency and Monetary on logarithmic scales).

4.4 Preprocessing

Preprocessing (FR4) applies the skew-gated transform of Section 2.2.1. Sample skewness (Eq. 2) is computed per feature; the six features exceeding the $|\gamma_1| > 0.5$ threshold Recency (0.89),Frequency (12.64), Monetary (25.34), AvgOrderValue (57.02), AvgInterPurchaseDays (1.45), and DistinctProducts (6.23) receive log1p (Eq. 3), while Tenure (0.39) is left on its

natural scale. All seven features are then standardized to z-scores (Eq. 4) and persisted together with the fitted scaler, so that any future data can be projected into exactly the same space. Figure 4.2 shows the before/after effect: the raw axes are dominated by a handful of extreme customers, while the transformed features are approximately symmetric on a common scale the correct geometry for the distance and density based algorithms that follow.

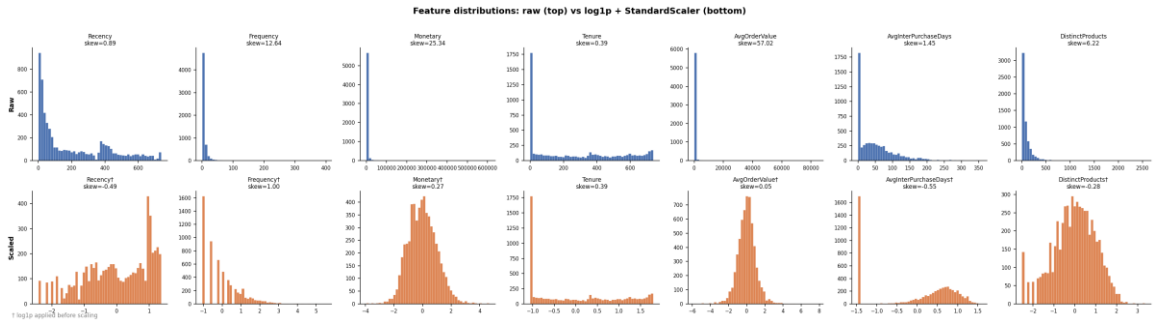

Figure 4.2 Feature distributions before (top) and after (bottom) the log1p + standardisation preprocessing.

4.5 Clustering: Six Algorithms

The clustering stage (FR5) fits all six algorithms of Section 2.2.2 behind a common interface, each with automatic hyper-parameter selection, and writes one cluster-label column per algorithm:

- **K-Means** (Eq. 5) sweeps k = 2…10. Silhouette analysis (Eq. 7) selects **k = 2** (silhouette 0.371, the maximum of the sweep), while geometric knee detection on the inertia curve [10] identifies **k = 4** (inertia 15,848 at the bend). This disagreement between two standard heuristics visible in Figure 4.4 and Table 4.3 is reported, not hidden: it is precisely why the multi index validation stage exists.
- **DBSCAN** reads its radius ε from the knee of the sorted 5 distance plot (Figure 4.6); min_samples = 5.
- **GMM** (Eq. 6) selects its component count by BIC over 2…10 components, choosing **10** (Figure 4.5) a warning sign, discussed in Section 5.2, that the likelihood keeps improving as components fragment.
- **HDBSCAN** uses min_cluster_size = 50 and min_samples = 5 deliberately requiring segments of at least ~1% of the customer base to be operationally meaningful.
- **Agglomerative** (Ward linkage) and **Spectral** (nearest neighbour affinity graph, 10 neighbours) reuse the K Means k = 2 for a like for like comparison at equal cluster count.

Figure 4.3 projects all six solutions onto the same two principal components (Eq. 11), which retain 76.6% of the variance (PC1 60.8%, PC2 15.9%), allowing direct visual comparison of the partition geometries.

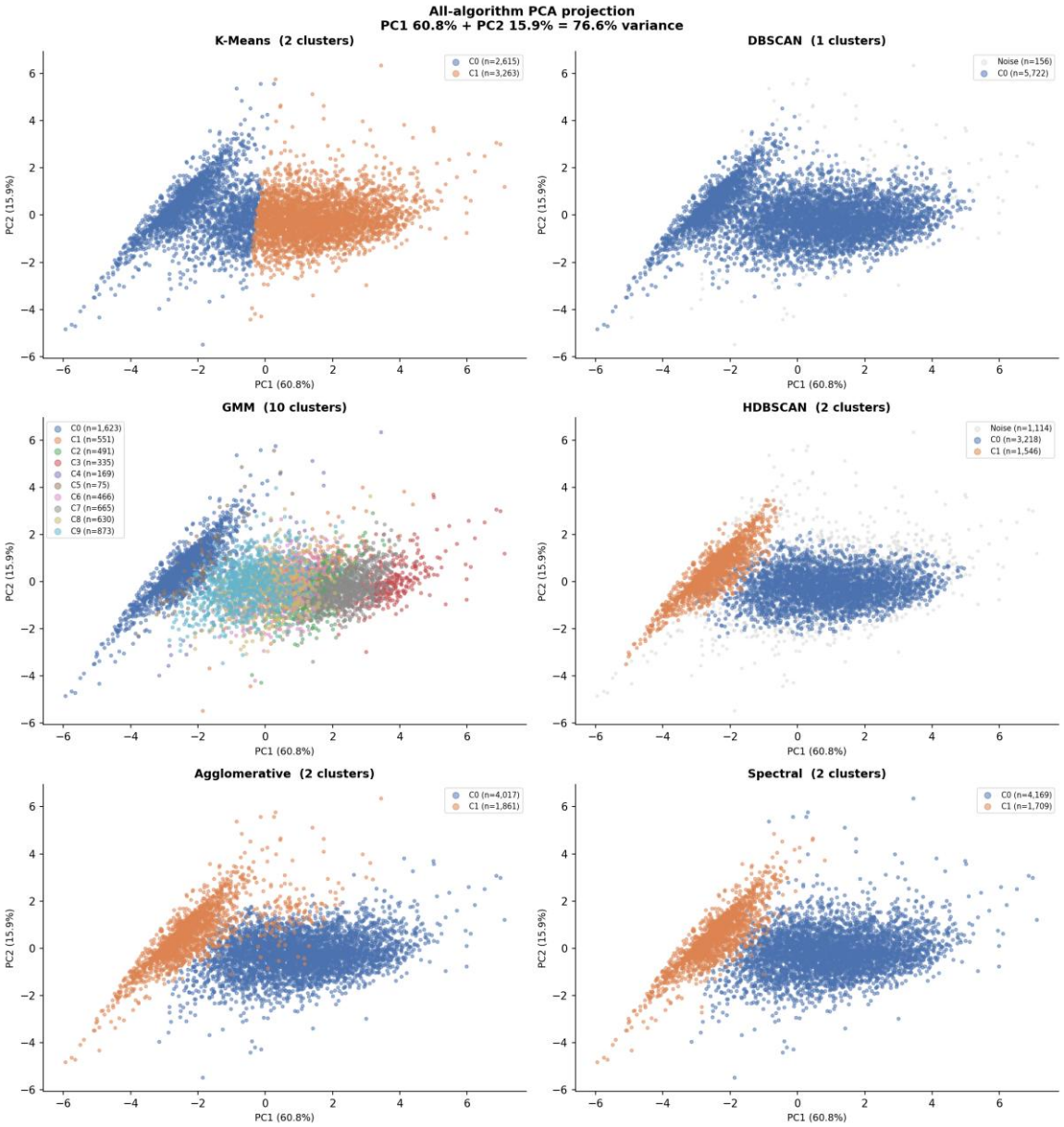

Figure 4.3 PCA projection of all six clustering solutions on identical axes (PC1 + PC2 ≈ 76.6% of variance).

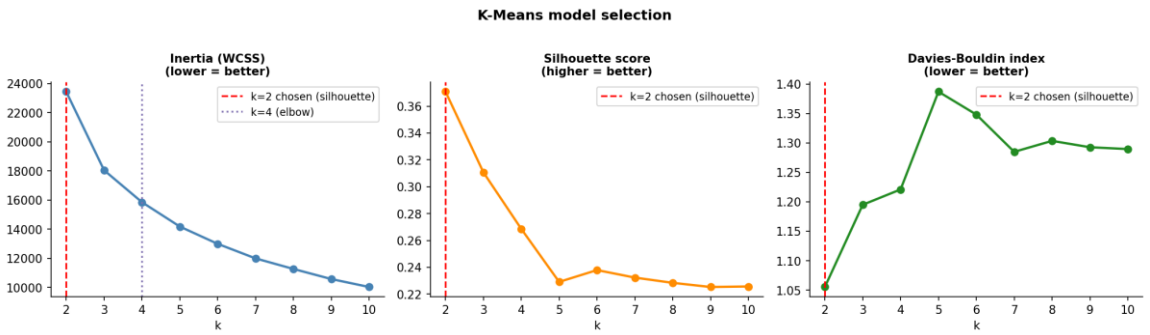

Figure 4.4 K Means model selection: inertia with the detected elbow (k = 4), silhouette (max at k = 2), and Davies Bouldin versus k.

Table 4.3 K-Means sweep metrics for k = 2…10: inertia (Eq. 5), silhouette (Eq. 7), and Davies Bouldin (Eq. 8).

k	inertia	silhouette	davies_bouldin
2	2.344e+04	0.3708	1.055
3	1.804e+04	0.3107	1.195
4	1.585e+04	0.2686	1.221
5	1.417e+04	0.2291	1.387
6	1.299e+04	0.2379	1.348
7	1.199e+04	0.2323	1.285
8	1.126e+04	0.2284	1.304
9	1.057e+04	0.2253	1.293
10	1.002e+04	0.2257	1.29

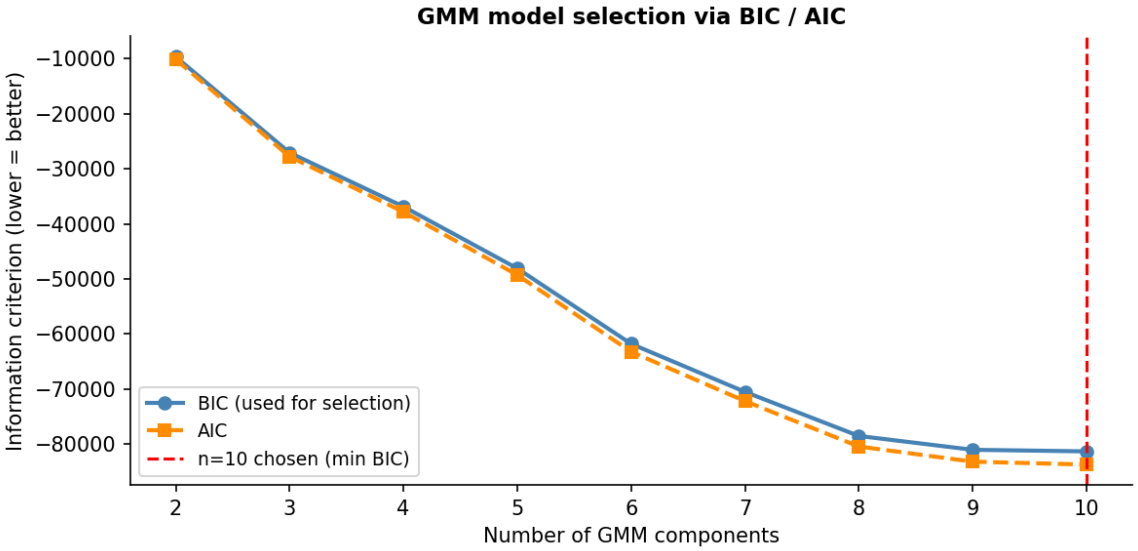

Figure 4.5 GMM model selection by BIC and AIC across component counts.

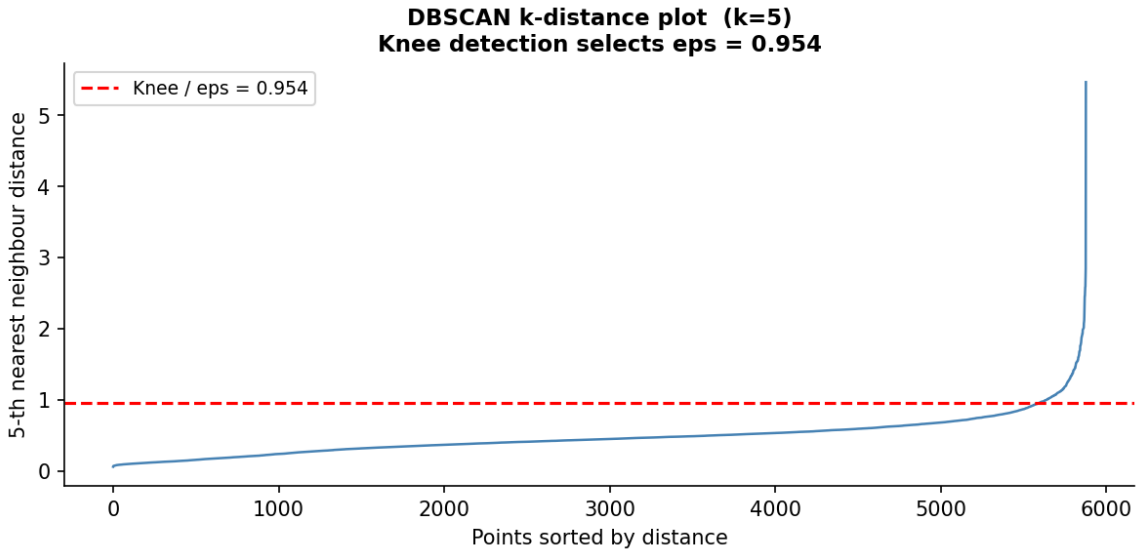

Figure 4.6 DBSCAN k-distance plot; the knee sets the neighbourhood radius ε.

4.6 Cluster Validation, Stability, and Selection

The validation stage (FR6) computes the three internal indices (Eqs. 7-9) for every solution excluding noise points for the density methods, so that indices measure the quality of the actual clusters and then measures stability by refitting each algorithm on **50 bootstrap resamples** and scoring each refit against the full data partition with the ARI (Eq. 10):

```
for algo in {K Means, DBSCAN, GMM, HDBSCAN, Agglomerative, Spectral}:
    labels_full = fit(algo, X)                      # partition on the full data
    for b in 1..50:
        idx  = resample(1..n, with_replacement, seed=42+b)
        lab_b = fit(algo, X[idx])                   # same hyper parameters
        ARI_b = adjusted_rand_index(labels_full[idx], lab_b)   # Eq. (10)
    stability[algo] = mean(ARI_1..ARI_50), std(ARI_1..ARI_50)
```

The per metric ranks are averaged into an overall ranking (Table 5.3), and the pre declared rule of Section 2.3 is applied in code, verbatim:

```
disqualify any solution with noise_fraction > 0.30
keep solutions with mean bootstrap ARI >= 0.70
among the survivors, select argmax(silhouette)
```

On this data the rule eliminates DBSCAN (mean ARI ≈ 0, having collapsed to a single cluster) and selects **HDBSCAN** (noise 18.95% under the cap; ARI above threshold; silhouette 0.416, the highest of all six). The full quantitative comparison and its interpretation are the subject of Section 5.2.

4.7 Segment Profiling and Naming

Profiling (FR7) translates the selected partition back into business language. Cluster means are computed in *original units* pounds and days, not z-scores and each cluster is named by a deterministic rule tree over its mean Recency, Frequency, and Monetary expressed as

multiples of the population mean (for example, a cluster with Frequency = 1, Tenure = 0 and high Recency is named "Lost Customers"). Determinism matters: the same profile always receives the same name, so names are reproducible across runs (NFR1).

HDBSCAN's solution comprises two genuine clusters plus the noise group, profiled in Table 4.4 and visualised in Figures 4.7 and 4.8:

- **General Customers** (cluster 0; n = 3,218; 54.8%): the mainstream moderate recency (mean 151 days), frequency ≈ 6.9, spend ≈ £2,422, long tenure (386 days).
- **Lost Customers** (cluster 1; n = 1,546; 26.3%): a sharply defined one time buyer group Frequency exactly 1.0, Tenure exactly 0, high recency (363 days), low spend (£297).
- **Noise / Uncategorised** (label −1; n = 1,114; 19.0%): *not* a failure but a finding the most valuable and most heterogeneous customers (mean spend £8,467, frequency ≈ 12.0, 115 distinct products), too atypical to force into either segment and flagged for individual treatment rather than generic campaigns.

The radar chart deliberately plots each segment's percentile against the full customer population rather than a min–max normalization across cluster means with only two genuine clusters, min to max would degenerately force every axis to 0% or 100% and mislead the reader.

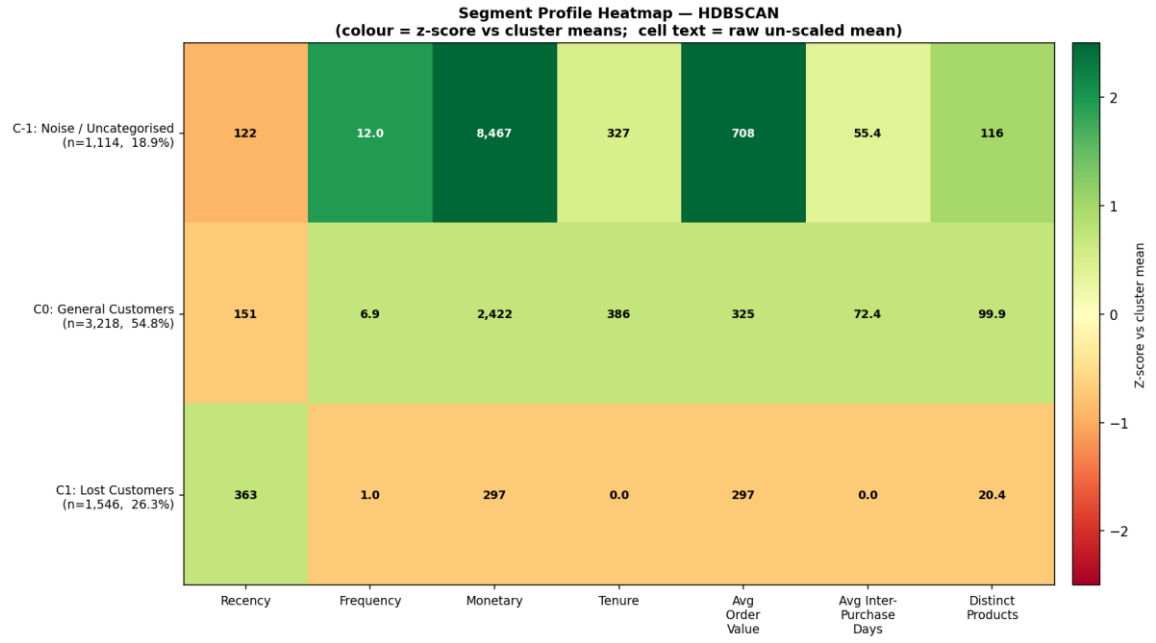

Figure 4.7 Segment profile heatmap for the selected HDBSCAN solution (colour = z-score versus cluster means; cell text = raw unscaled means).

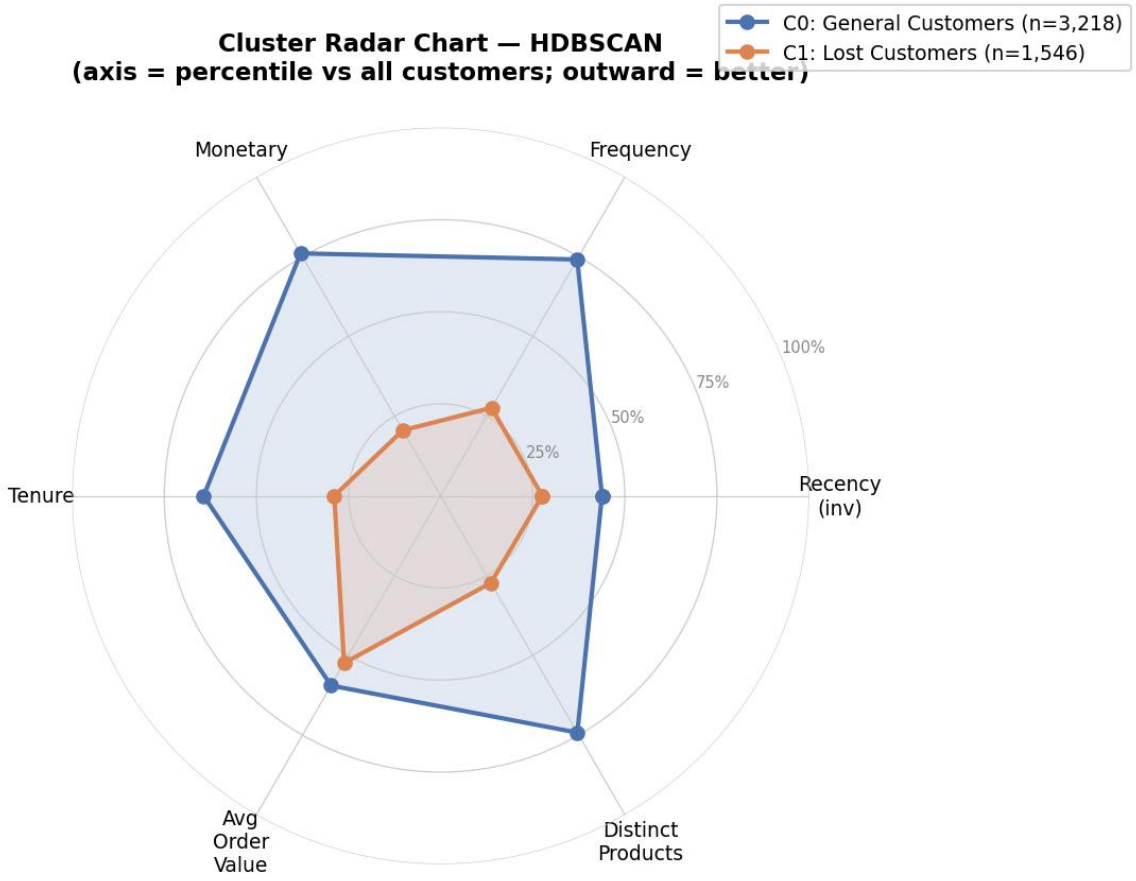

Figure 4.8 Segment radar chart: each axis is the segment's percentile against the full customer population (outward = better).

Table 4.4 HDBSCAN segment profiles: mean features in original units, size, share, and assigned marketing name.

HDBSCAN_ Cluster	Recency	Frequency	Monetary	Tenure	AvgOrder Value	AvgInterPurchaseDays	DistinctProducts	size	pct_of_ total	segment name
-1	121.6	11.98	8,467	326.6	707.6	55.42	115.9	1,114	18.95	Noise / Uncategorised
0	151.4	6.862	2,422	385.6	324.7	72.42	99.86	3,218	54.75	General Customers
1	362.8	1	297.1	0	297.1	0	20.38	1,546	26.3	Lost Customers

4.8 Customer Lifetime Value

The CLV stage (FR8) fits the BG/NBD model (Eqs. 12–13) and the Gamma Gamma model (Eq. 14) on the per customer frequency / recency / tenure / monetary summary using the `lifetimes` library, both with a small L2 penaliser (0.001) for numerical stability. The Gamma Gamma independence assumption is checked before fitting: the correlation between transaction frequency and mean transaction value is near zero on this data, so the model is admissible. A defensive guard clips the model's baseline monetary estimate when the fitted

shape parameter q ≤ 1 (where the population mean pv/(q−1) in Eq. 14 is undefined or negative). The stage outputs, per customer: P(alive) (Eq. 12), expected purchases over 90/180/365 days (Eq. 13), expected transaction value (Eq. 14), and the 12-month CLV discounted at 1% per month (Eq. 15).

The results reproduce the classic long tail of retail value: mean CLV £1,418 against a median of £371, a maximum of £323,792, and a total 12-month portfolio value of **£8.33 million**. Mean P(alive) is 0.909 (median 0.980): most of the retained base is very probably still active, but a decisive minority is not exactly the signal the notification engine needs. Figure 4.9 shows the CLV distribution and the engagement-versus-retention scatter.

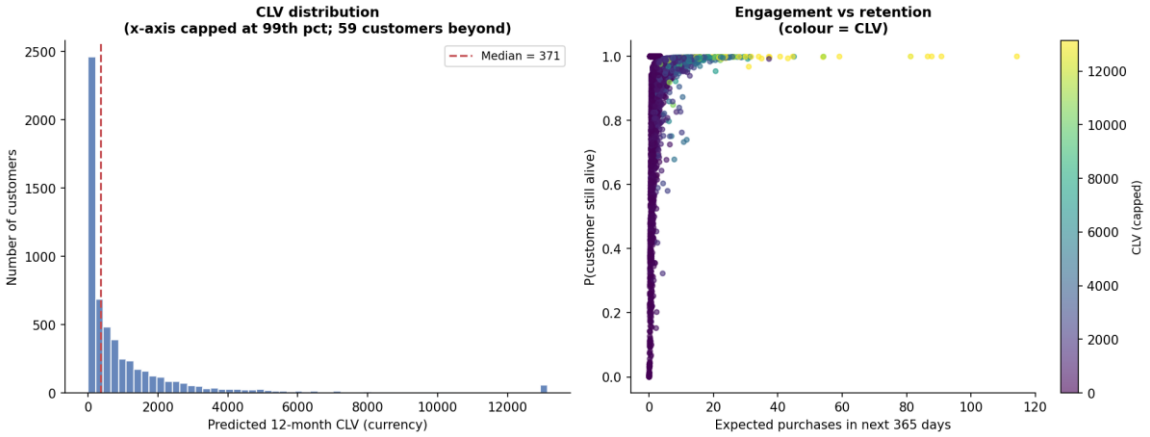

Figure 4.9 CLV distribution (left) and the frequency–recency scatter coloured by CLV (right).

Table 4.5 Summary statistics of the CLV model outputs across all 5,878 customers.

	clv	prob_alive	exp_avg_value	pred_purchases_90d	pred_purchases_180d	pred_purchases_365d
count	5,878	5,878	5,878	5,878	5,878	5,878
mean	1,418	0.9095	322.7	0.7981	1.585	3.175
std	7,731	0.1731	2,279	1.216	2.42	4.863
min	0	3.3e-10	0	1.4e-09	2.7e-09	5.5e-09
25%	0	0.924	0	0.1741	0.3461	0.6938
50%	370.7	0.9799	217.8	0.4563	0.9051	1.805
75%	1,224	1	380.2	0.9736	1.934	3.877
max	3.238e+05	1	1.702e+05	28.49	56.76	114.3

4.9 Churn Classification

The churn stage (FR9) implements the design of Section 2.2.6 exactly. The label thresholds Recency at its 90th percentile (Eq. 16) **535 days** on this data marking **587 customers (10.0%)** as churned: a realistic minority class. Because the label is a function of Recency, **Recency is excluded from the feature matrix**; the models see the six remaining

behavioural features. Six classifiers are trained on a stratified 80/20 hold out split with 5 fold stratified cross validation: logistic regression (Eq. 18), random forest and decision tree (Gini splits, Eq. 19) with class weighting (Eq. 17), XGBoost with the equivalent `scale_pos_weight`, gradient boosting, and k nearest neighbours (k = 15, distance weighted). Gradient boosting and KNN are *deliberately* left without imbalance handling a controlled contrast whose consequences, catastrophic recall collapse despite competitive ROC AUC, are a substantive finding of the evaluation (Section 5.3). The full model configuration:

Model	**Key hyper parameters**	**Imbalance handling**
LogisticRegression	max_iter = 1000	class_weight = "balanced" (Eq. 17)
RandomForest	300 trees, default depth	class_weight = "balanced"
XGBoost	300 trees, depth 5, lr 0.05, subsample 0.8	scale_pos_weight = N_0/N_1
GradientBoosting	200 trees, depth 3, lr 0.1, subsample 0.8	none (deliberate contrast)
DecisionTree	max_depth = 5	class_weight = "balanced"
KNN	k = 15, distance-weighted vote	none (deliberate contrast)

All models share the same stratified 80/20 split, the same 5 fold stratified cross validation, and the same six leakage guarded features, so every observed difference in Section 5.3 is attributable to the model, not the protocol.

Feature importances from the best tree model (Figure 4.10) rank **Tenure (0.370)** and **AvgInterPurchaseDays (0.219)** as the dominant churn signals, ahead of Monetary (0.138) and Frequency (0.131) an operationally intuitive result: customers who never settled into a stable buying rhythm are the ones most likely to lapse, and the guard against the trivially leaky signal (Recency) forces the models to learn these genuine behavioural precursors.

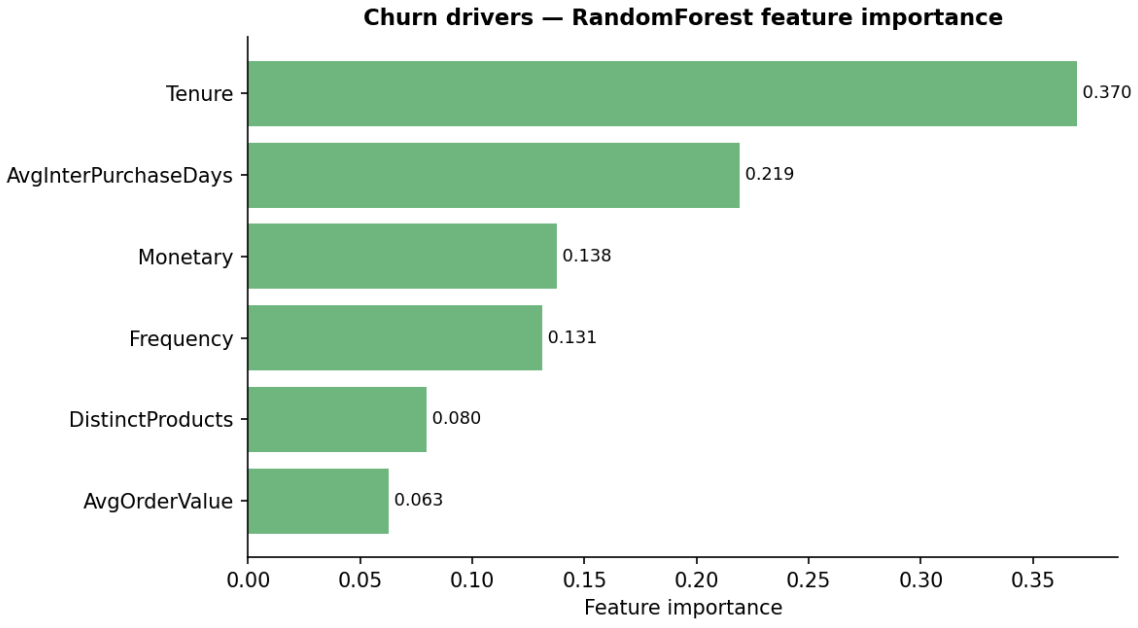

Figure 4.10 Churn feature importances from the best tree model. Recency is absent by design it defines the label (leakage guard).

4.10 Year-on-Year Segment Migration

The migration stage (FR10) answers a question a single static segmentation cannot: *do the segments describe stable behavioural states, and how do customers move between them?* Each trading year is segmented independently RFM is recomputed within the year and the same deterministic naming rules applied and the two segmentations are joined on Customer ID for the **2,772 customers active in both years**. The result is a transition matrix in both raw counts and row normalized rates (Tables 5.8–5.9, Figure 5.4), analysed in Section 5.5. The design choice to re segment per year (rather than re using the global two year clusters) matters: migration is only meaningful between states *defined by the same rules applied to comparable windows*.

4.11 Rule-Based Notification Engine

The notification engine (FR11) is where analysis becomes action. It composes three inputs per customer the HDBSCAN segment name (Section 4.7), a CLV tier, and a churn risk band into a concrete campaign instruction, in three steps.

Step 1 — segment playbook. Each segment name maps to a baseline campaign: an action, a channel, an offer, and a base priority (1–5). The full playbook covers all eight names the deterministic naming rules of Section 4.7 can produce (the per-year migration segmentation of Section 4.10 produces the finer-grained names; the global HDBSCAN solution activates three):

Segment	Action	Channel	Base priority
Champions	VIP loyalty reward + early access	Email + App push	4
Loyal Customers	Cross-sell + loyalty tier upgrade	Email + App push	3
Big Spenders	Premium product recommendations	Email + Personal outreach	4
Potential Loyalists	Frequency-building nudge	App push + Email	3
General Customers	Engagement / category promotion	Email	2
At Risk	Win-back outreach	Email + SMS	5
Lost Customers	Reactivation campaign	Email	2
Noise / Uncategorized	Standard newsletter	Email	1

The noise entry is deliberately the *least* aggressive: customers the model cannot place here, often the highest value accounts (Section 4.7) should not be blasted with generic offers.

Step 2 — value and risk modulation. The playbook baseline is then escalated by the customer's own predictions. CLV tiers split at the 80th percentile (High) and 40th percentile (Low) of the CLV distribution; churn bands split at probabilities 0.50 (High) and 0.25 (Medium). High churn risk raises priority by 2 and, for High/Medium value customers, overrides the action to **"Priority retention intervention"** on the premium channel mix (Email + SMS + Personal outreach); for Low-value high-risk customers the action becomes **"Low-cost automated reactivation"** spend is concentrated where value is at risk. Medium risk raises priority by 1; high value raises it by 1 more; the total is capped at 5:

$$priority = \min(base\_priority + 2 \cdot [churn\ High] + 1 \cdot [churn\ Medium] + 1 \cdot [CLV\ High], 5) \quad (28)$$

Step 3 — contact timing. Rather than a fixed campaign drumbeat, the recommended contact day is a fraction (0.6) of the customer's own average inter-purchase interval reaching them *before* their natural next-purchase window closes with a 30 day default for one time buyers whose cadence is undefined:

$$contact\_days = max(round(0.6 \cdot AvgInterPurchaseDays), 1), \text{ default 30 if cadence unknown} \quad (29)$$

Applied to the full base, the engine plans campaigns for all 5,878 customers: 3,023 engagement promotions, 1,653 low-cost automated reactivations, 977 standard newsletters, 161 reactivation campaigns, and **64 priority retention interventions** the escalation path is rare by construction, reserved for the customers who are simultaneously valuable and at risk. The engine also exposes `recommend(customer_id)`, which computes the same decision on demand for a single customer; this function powers the dashboard's live lookup page. Table 4.6 shows the head of the persisted plan.

Table 4.6 Per-customer notification plan (first 10 of 5,878 rows): segment, CLV tier, churn band, and the resulting action, channel, offer, priority, and contact timing.

Customer ID	segment	clv	clv_tier	churn_probability	churn_risk	action	channel	offer	priority	recommended_contact_days
15,398	General Customers	2,549	High	0.5609	High	Priority retention intervention	Email + SMS + Personal outreach	High value personalised retention offer	5	19

Customer ID	segment	clv	clv_tier	churn_probability	churn_risk	action	channel	offer	priority	recommended_contact_days
12,689	General Customers	1,977	High	0.6201	High	Priority retention intervention	Email + SMS + Personal outreach	High value personalised retention offer	5	30
14,418	General Customers	4,361	High	0.3159	Medium	Engagement / category promotion	Email	Seasonal category promotion	4	21
13,644	General Customers	4,040	High	0.2816	Medium	Engagement / category promotion	Email	Seasonal category promotion	4	13
12,518	General Customers	3,717	High	0.2978	Medium	Engagement / category promotion	Email	Seasonal category promotion	4	13
13,994	General Customers	3,421	High	0.3366	Medium	Engagement / category promotion	Email	Seasonal category promotion	4	14
15,977	General Customers	3,395	High	0.3017	Medium	Engagement / category promotion	Email	Seasonal category promotion	4	6
14,387	General Customers	3,065	High	0.2565	Medium	Engagement / category promotion	Email	Seasonal category promotion	4	15
12,646	Noise / Uncategorised	2,783	High	0.5777	High	Priority retention intervention	Email + SMS + Personal outreach	High-value personalised retention offer	4	13
13,437	General Customers	2,780	High	0.3119	Medium	Engagement / category promotion	Email	Seasonal category promotion	4	15

(Showing first 10 of 5878 rows.)

4.12 Monte Carlo ROI Simulation

The ROI stage (FR12) implements Section 2.2.8. Customers are grouped by planned action; each action carries a documented prior response rate encoded as the mean of a Beta prior with concentration 150 (Eq. 24) reflecting the planning judgement that high-touch, tightly targeted interventions respond better than mass sends:

Campaign action	Prior response rate
Priority retention intervention	0.30
VIP loyalty reward + early access	0.25
Premium product recommendations	0.20
Cross-sell + loyalty tier upgrade	0.15
Win-back outreach	0.12
Frequency-building nudge	0.10
Engagement / category promotion	0.08
Reactivation campaign	0.05
Low-cost automated reactivation	0.03
Standard newsletter	0.02

One simulated world is computed as:

```
for sim in 1..10,000:
    profit = 0
    for each action group g:                    # N_g customers, prior p_g
        r_g ~ Beta(p_g * 150, (1 - p_g) * 150)  # response-rate uncertainty
        k_g ~ Binomial(N_g, r_g)                # conversions
        m_g ~ Normal(1, 0.20), clipped > 0      # basket-size noise
        revenue_g = k_g * AOV_g * margin(30%) * (1 - discount(10%)) * m_g
        cost_g    = N_g * channel_cost(g)       # Email .05 / push .02 / SMS .15 /
personal 5.00
        profit   += revenue_g - cost_g
    ROI[sim] = profit / total_cost              # Eq. (26)
repeat identically for the static baseline (all 5,878 by email, response 2%)
report mean, median, 95% credible interval, P(ROI > 0), uplift  # Eq. (27)
```

Each of **10,000** simulated worlds draws a response rate per group, conversions from the induced Binomial, and revenue per conversion as the group's mean order value under 30% gross margin and 10% promotional discount, perturbed by multiplicative Gaussian noise (σ = 0.20, clipped positive) (Eq. 25). Costs are deterministic: per-contact channel costs (Email £0.05, App push £0.02, SMS £0.15, personal outreach £5.00) summed over each group's channel mix a total campaign cost of **£623.50**. The same machinery simulates the static baseline: every customer contacted once, by email, with one generic offer at a 2% blanket response rate (cost £293.90). Because both plans share the same simulation machinery and noise assumptions, their *difference* (Eq. 27) is far more robust than either absolute figure this is the deliberate design of the commercial evaluation, whose results Section 5.7 reports.

4.13 Interactive Dashboard

The dashboard (FR13) is a Streamlit application of eight pages, each reading only persisted artefacts (Section 3.2.2):

Page	What the user sees	Reads
Overview	headline metrics (customers, portfolio CLV, churn rate, campaigns) + pipeline map	features, CLV, churn, plan

Page	What the user sees	Reads
Segments	segment sizes, un-scaled profile table, heatmap, radar	segment profiles + figures
Lifetime Value	CLV distribution, summary statistics	Customer clv, clv summary
Churn Risk	model comparison table, ROC/PR curves, risk histogram	churn tables + figures
Migration	Year on year transition heatmap	migration tables + figure
Notifications	the full 5,878 row plan, filterable by segment/risk/priority	notification plan
ROI Simulation	ROI and profit distributions, full summary	Roi simulation summary + figure
Customer Lookup	live recommendation for any Customer ID (12346–18287)	recommend() API

Figure 4.11 shows the Overview page with the real headline numbers (5,878 customers; £8,333,122 portfolio CLV; 10.0% churn rate); Figure 4.12 shows the Segments page; Figure 4.13 shows the ROI page with the simulated distributions and the full summary table. The dashboard turns the batch pipeline into a tool a non-technical marketing user can operate: the lookup page in particular answers, for any Customer ID between 12346 and 18287, "what should we send this person, and why?" with the *why* fully traceable to the rules of Section 4.11 (NFR5).

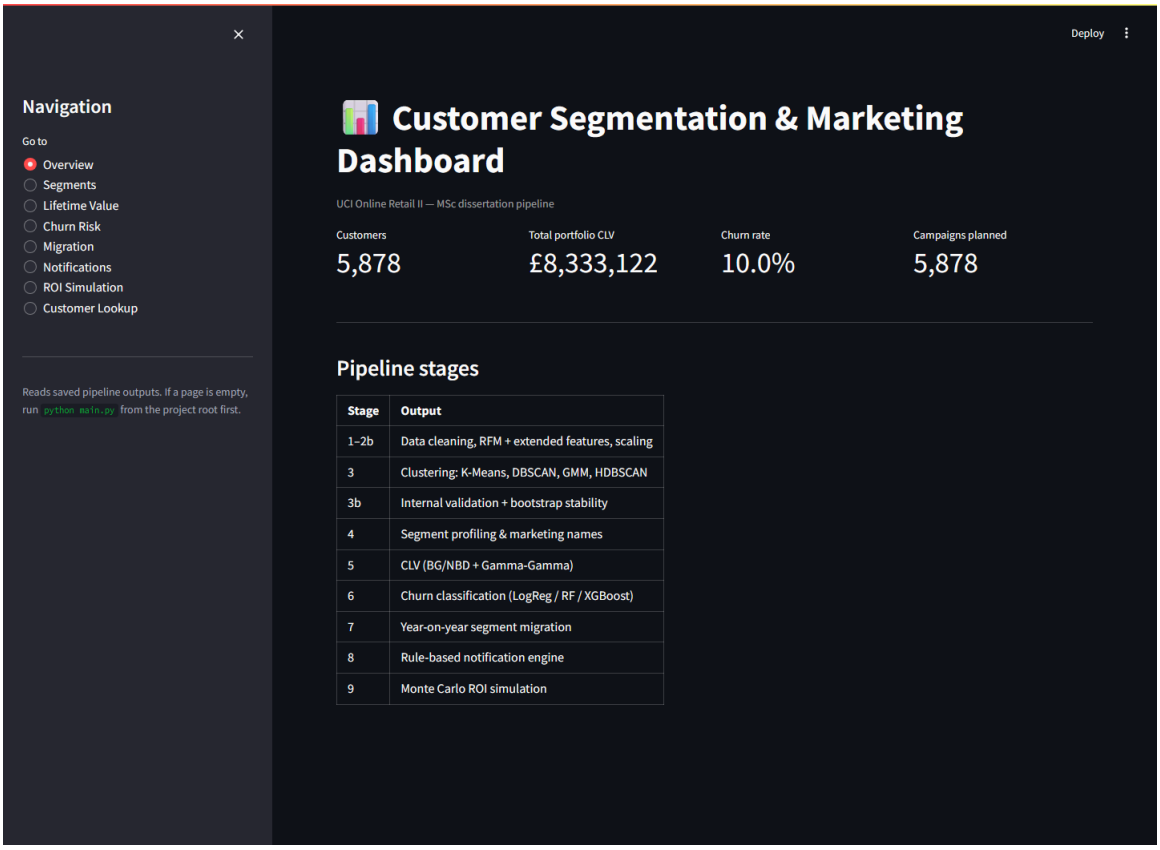

Figure 4.11 Dashboard Overview page: navigation sidebar, headline metrics (5,878 customers; £8,333,122 portfolio CLV; 10.0% churn rate), and pipeline summary.

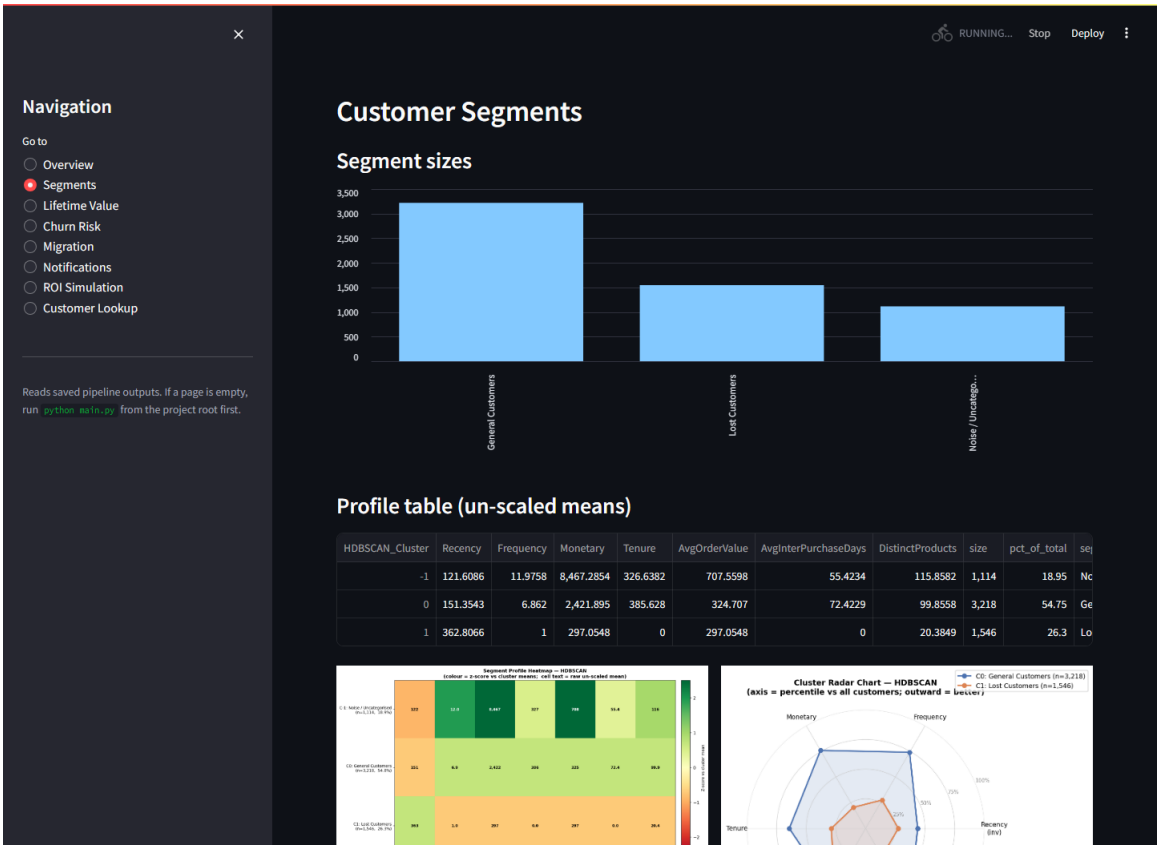

Figure 4.12 Dashboard — Segments page: segment sizes, the un-scaled profile table, and the profile heatmap and radar.

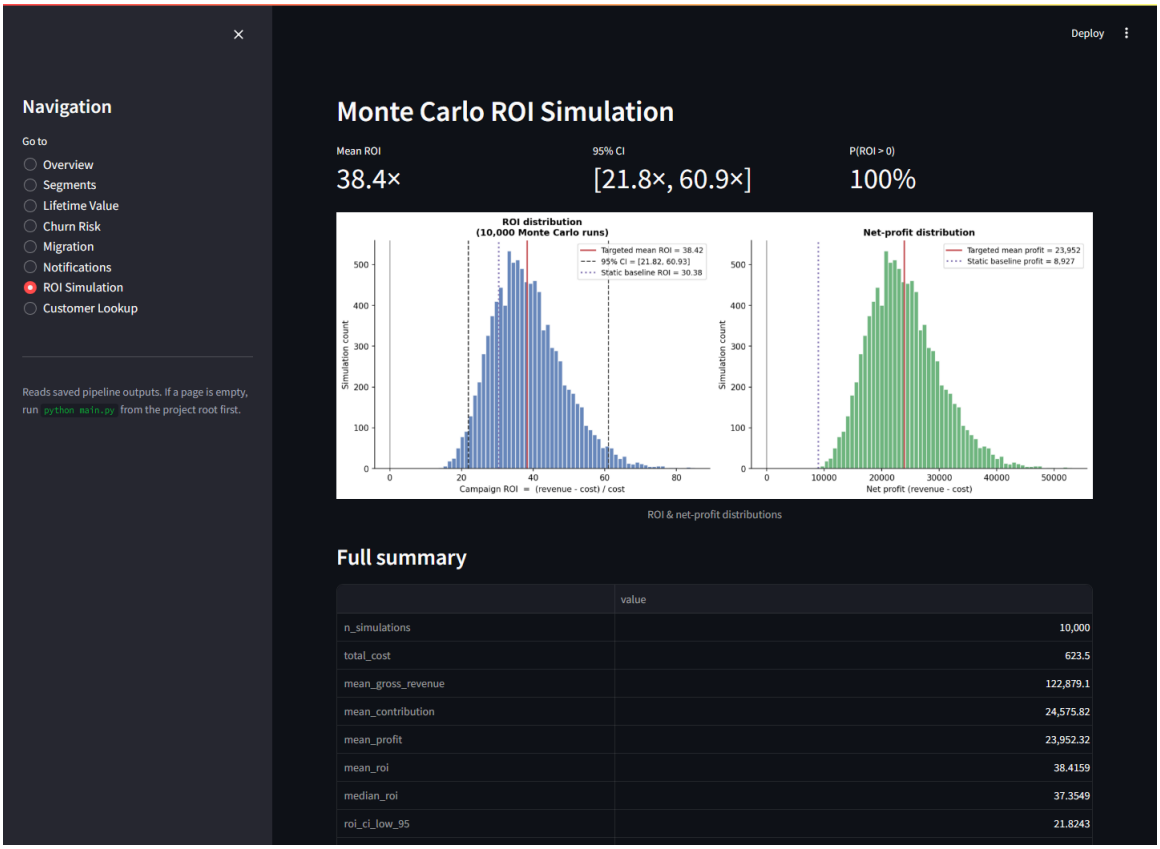

Figure 4.13 Dashboard ROI Simulation page: headline metrics (mean ROI 38.4×; 95% CI [21.8×, 60.9×]; P(ROI>0) = 100%), the simulated ROI and net profit distributions with the static baseline overlaid, and the full summary table.

Chapter 5
Software Testing and Evaluation

This chapter first reports the software testing methodology and its outcome (Section 5.1), then evaluates the system layer by layer against the objectives: the segmentation (5.2), the churn models (5.3), the CLV model (5.4), the migration analysis (5.5), the notification plan (5.6), and the commercial ROI (5.7). Section 5.8 traces every functional requirement to its evidence, and Section 5.9 states the threats to validity.

5.1 Software Testing

5.1.1 Methodology

Testing follows three complementary strands. First, an **automated unit-test suite** of **20 pytest tests across seven modules** exercises the core logic on small synthetic datasets constructed inside the tests themselves deliberately independent of the raw data file, so the tests are fast, deterministic, and verify *logic* rather than data (NFR6). Table 5.1 summarizes what each module's tests assert.

Table 5.1 Unit test suite: modules, test counts, and the logic each verifies (20 tests, all passing; runtime < 3 s).

Test module	Tests	What is verified
test_cleaning.py	2	Each cleaning rule removes exactly the intended rows; the audit table records correct counts and order (FR2).
test_features.py	2	RFM derivations against hand-computed values on a toy ledger; Frequency counts distinct invoices, not lines (FR3).
test_preprocessing.py	3	Skew detection triggers log1p only above threshold (Eq. 2); scaled output has mean ≈ 0, std ≈ 1 (Eq. 4); scaler roundtrips (FR4).
test_churn.py	2	The label matches the 90th-percentile rule (Eq. 16); Recency is absent from the model feature matrix the leakage guard holds (FR9).
test_migration.py	3	Per-year segmentation joins correctly on Customer ID; transition rates row normalize to 1 (FR10).
test_notifications.py	5	Playbook lookups; priority escalation and its cap (Eq. 28); high risk/high value override; low-value path; contact-day rule (Eq. 29) (FR11).
test_roi.py	3	Simulation reproducibility under the fixed seed; ROI arithmetic (Eq. 26); static baseline uplift sign and magnitude (Eq. 27) (FR12).

Second, **reproducibility testing** (NFR1): the full pipeline was executed repeatedly and the result tables compared byte for byte identical on every run, as guaranteed by the fixed seed and pinned snapshot date. Third, **static analysis**: the codebase is lint clean under pyflakes, and dead code identified during development (unused imports, an unused backwards-compatibility alias) was removed rather than suppressed.

5.1.2 Outcome

All 20 tests pass (a 100% pass rate; the suite completes in under three seconds, so it was executed after every change). The end-to-end pipeline completes in ≈160 seconds on commodity CPU hardware (NFR4), and each stage can be re run in isolation from its persisted inputs (NFR2). Beyond the automated suite, every result table and figure in this report was regenerated from scratch after the final code change the numbers quoted in Chapters 4 and 5 *are* the current pipeline output, not survivors of an earlier version.

5.2 Evaluation of the Segmentation

5.2.1 Quantitative comparison

Table 5.2 reports the three internal indices (Eqs. 7–9) for all six algorithms; Table 5.3 the overall ranking after adding bootstrap stability (Eq. 10); Figure 5.1 the stability distributions across the 50 bootstrap resamples.

Table 5.2 Internal validity metrics for all six clustering algorithms (noise excluded from index computation for density methods).

Algorithm	N clusters	Noise fraction	silhouette	Davies Bouldin	Calinski_harabasz
K-Means	2	0	0.3708	1.055	4,439
DBSCAN	1	0.0265			
GMM	10	0	0.0809	3.809	989.6
HDBSCAN	2	0.1895	0.4157	0.89	4,007
Agglomerative	2	0	0.3446	1.101	3,188
Spectral	2	0	0.3658	0.9842	3,587

Table 5.3 Overall algorithm ranking: per-metric ranks averaged across silhouette, Davies–Bouldin, Calinski–Harabasz, and mean bootstrap ARI.

Algorithm	silhouette rank	Davies Bouldin rank	Calinski harabasz rank	ARI mean rank	mean rank	overall rank
HDBSCAN	1	1	2	3	1.75	1
K-Means	2	3	1	2	2	2
Spectral	3	2	3	1	2.25	3
Agglomerative	4	4	4	4	4	4
GMM	5	5	5	5	5	5
DBSCAN	6	6	6	6	6	6

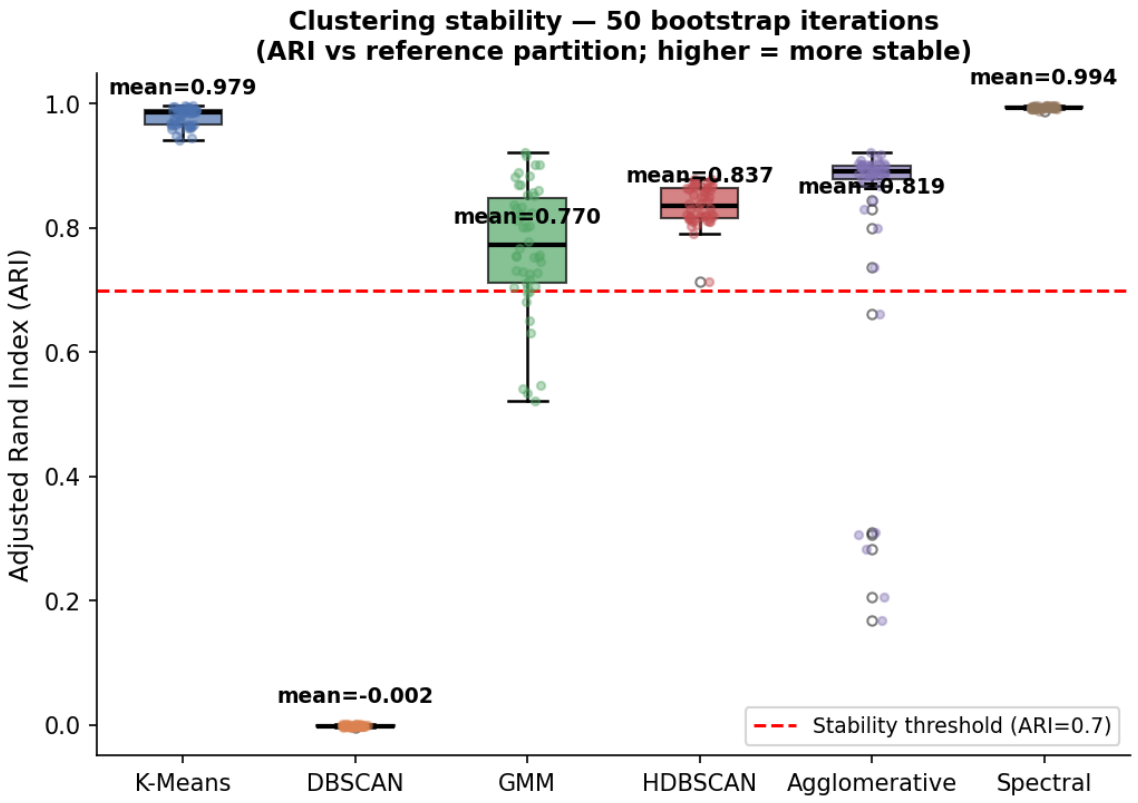

Figure 5.1 Bootstrap stability: distribution of the Adjusted Rand Index across 50 resamples per algorithm.

The ranking places **HDBSCAN first** (mean rank 1.75), then K-Means (2.00), Spectral (2.25), Agglomerative (4.00), GMM (5.00), and DBSCAN last (6.00, worst on every metric). HDBSCAN achieves the best silhouette (0.416) and best Davies Bouldin (0.890) precisely because it declines to force genuinely unassignable customers into clusters 18.95% are labelled noise, under the 30% disqualification cap and Section 4.7 showed that this noise group is operationally meaningful, not discarded data.

5.2.2 Interpretation of the failures and disagreements

Three negative results are as informative as the winner. **DBSCAN failed outright**, collapsing to a single cluster with near zero bootstrap ARI: with one global radius it cannot accommodate this data's smoothly varying density, and the pre-declared ARI ≥ 0.70 rule eliminates it mechanically the selection framework did its job. **GMM over fragmented**, with BIC selecting 10 components whose silhouette is only 0.081 and Davies Bouldin 3.81: the likelihood complexity trade off (Eq. 6) keeps rewarding finer Gaussian mixtures on a distribution that is not a mixture of well-separated Gaussians a caution against selecting clustering structure by likelihood criteria alone. **The K Means heuristics disagreed** (elbow k = 4 vs silhouette k = 2, Section 4.5), confirming that any single selection heuristic is fragile, which is the argument for the multi index, multi algorithm validation this project performs.

Finally, **Spectral clustering was the most stable** solution (mean ARI ≈ 0.99) but did not improve separation over K-Means stability and separation are different qualities, and a defensible selection needs both, which is exactly how the selection rule is ordered.

5.2.3 Relation to prior findings

The outcome is consistent with the comparative literature rather than surprising against it. The no dominant algorithm finding of [20], [21], [40] predicts exactly what Tables 5.2–5.3 show: the centroid, hierarchical, and graph families land close together on this data while the density family splits to both extremes (HDBSCAN best, DBSCAN worst), because the decisive dataset property is smoothly varying density with a genuinely unassignable minority a property HDBSCAN's hierarchy absorbs and DBSCAN's single radius cannot. The segment structure itself echoes Chen et al. [15], who also recover a dominant mainstream group and a sharply defined lapsed group on this dataset family; the addition here is that the structure arrives with quantified validity, measured stability, and an explicit account of the fifth of the base that fits no segment the part a marketing team would otherwise have mistargeted.

5.2.4 Verdict against O3

Objective O3 asked for benchmarked algorithms, principled selection, and validated clusters. The evidence: six algorithms fitted with automatic hyper-parameters (Section 4.5), three validity indices plus 50-round bootstrap stability computed for each (Tables 5.2–5.3), and a pre-declared three-step rule whose application is mechanical and reproducible (Section 4.6). The resulting segmentation is coarse two clusters plus noise but honest: on this feature space, that is the structure the evidence supports, and forcing more clusters (as GMM does) demonstrably degrades quality.

5.3 Evaluation of the Churn Models

5.3.1 Quantitative comparison

Table 5.4 reports hold out and cross validated metrics with full confusion counts for all six models; Table 5.5 the multi metric ranking; Figure 5.2 the ROC and precision recall curves; Table 5.6 the pairwise McNemar p values (Eq. 23).

Table 5.4 Churn model comparison on the stratified hold-out set: ROC AUC, PR AUC, precision, recall, F1 (Eqs. 20–22), 5 fold cross-validated ROC AUC, and confusion counts.

Model	ROC AUC	PR AUC	Precision	Recall	F1	CV ROC AUC mean	CV ROC AUC std	TN	FP	FN	TP
LogisticRegression	0.8459	0.2795	0.2296	0.8889	0.3649	0.8459	0.0131	710	349	13	104

Model	ROC AUC	PR AUC	Precision	Recall	F1	CV ROC AUC mean	CV ROC AUC std	TN	FP	FN	TP
RandomForest	0.849	0.2874	0.2708	0.7521	0.3982	0.8424	0.0108	822	237	29	88
XGBoost	0.8469	0.2972	0.2663	0.735	0.3909	0.8398	0.0074	822	237	31	86
GradientBoosting	0.8461	0.2851	0.2105	0.0342	0.0588	0.8351	0.0078	1,044	15	113	4
DecisionTree	0.828	0.2544	0.2338	0.9231	0.3731	0.8218	0.0212	705	354	9	108
KNN	0.807	0.2483	0.2162	0.0684	0.1039	0.8163	0.0154	1,030	29	109	8

Table 5.5 Overall churn-model ranking across ROC-AUC, PR-AUC, F1, and cross-validated ROC-AUC.

Model	ROC AUC rank	PR AUC rank	F1_rank	CV ROC AUC mean rank	Mean rank	Overall rank
RandomForest	1	2	1	2	1.5	1
XGBoost	2	1	2	3	2	2
LogisticRegression	4	4	4	1	3.25	3
GradientBoosting	3	3	6	4	4	4
DecisionTree	5	5	3	5	4.5	5
KNN	6	6	5	6	5.75	6

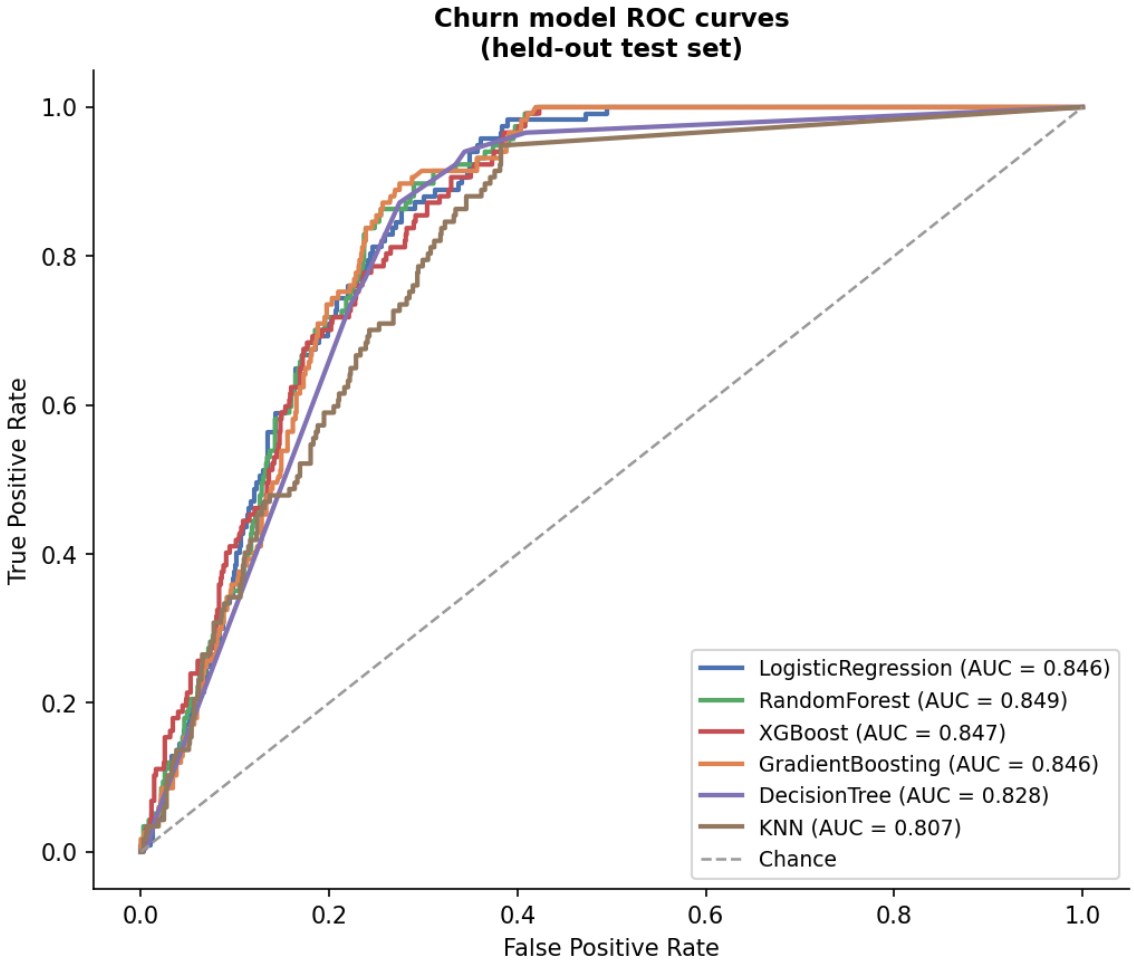

Figure 5.2a ROC curves for the six churn models.

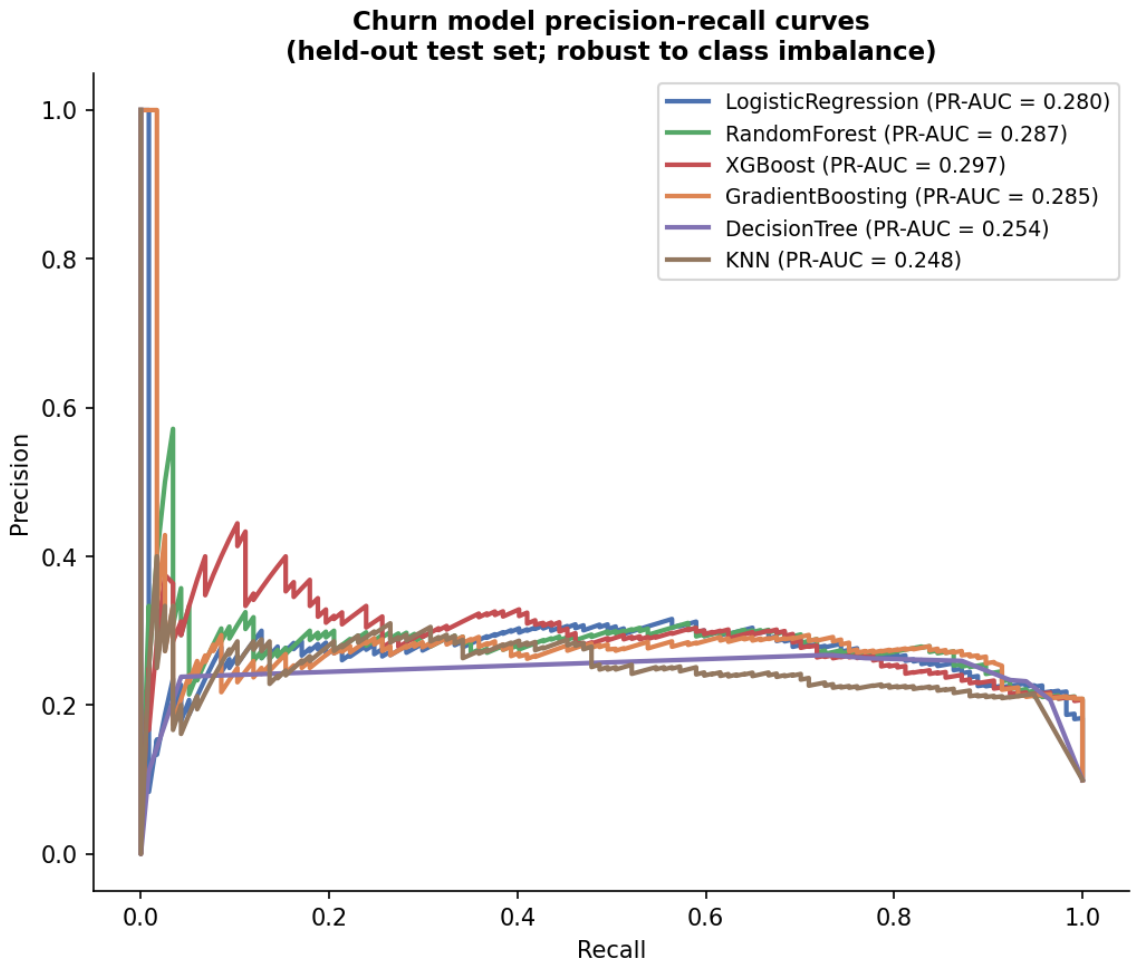

Figure 5.2b Precision recall curves for the six churn models the more discriminating view under 10% class imbalance.

Table 5.6 Pairwise McNemar exact p-values between the six churn models (Eq. 23). Values above 0.05 mean the two models are statistically indistinguishable on this test set.

	LogisticRegression	RandomForest	XGBoost	GradientBoosting	DecisionTree	KNN
LogisticRegression	1	6.1e-18	2.1e-15	2.2e-30	1	3.1e-29
RandomForest	6.1e-18	1	0.8776	1.6e-15	2.7e-17	2.7e-14
XGBoost	2.1e-15	0.8776	1	5.9e-16	2.1e-15	8.1e-15
GradientBoosting	2.2e-30	1.6e-15	5.9e-16	1	4.0e-30	0.1539
DecisionTree	1	2.7e-17	2.1e-15	4.0e-30	1	7.6e-29
KNN	3.1e-29	2.7e-14	8.1e-15	0.1539	7.6e-29	1

5.3.2 Findings

The headline is honest. The best model, Random Forest, reaches ROC-AUC 0.849 (CV mean 0.842 ± 0.011) with recall 0.752 and precision 0.271. A value near 1.0 would have been a red flag for leakage; 0.85 with the defining feature excluded is a credible ceiling for six behavioral features, and the tight cross-validation spread shows it is not a lucky split.

The two best models are statistically tied. Random Forest edges XGBoost on the ranking (Table 5.5), but McNemar's exact test gives p = 0.878 for that pair (Table 5.6): their disagreements on the test set are symmetric noise, not systematic superiority. A naive "Random Forest wins" claim would overstate the evidence — the defensible statement, and the one this project makes, is that *either* ensemble is an acceptable deployment choice.

ROC-AUC alone would have misled. Gradient Boosting posts a competitive ROC AUC of 0.846 and a recall of 0.034: at the default threshold it identifies just 4 of 117 hold out churners (TP = 4, FN = 113). KNN behaves the same way (recall 0.068). Both were deliberately left without imbalance handling (Section 4.9), and both collapse to near-trivial predictors in exactly the way Section 2.1.4 warned; their McNemar row ($p \approx 0.15$ against each other, $p < 10^{-13}$ against every properly weighted model) confirms the failure is systematic. The class weighted models logistic regression (recall 0.889), decision tree (0.923), random forest (0.752), XGBoost (0.735) all retain useful recall. This contrast, engineered into the experiment, is the project's clearest demonstration that metric choice and imbalance handling are not bureaucratic details but the difference between a usable and a useless churn model.

The models learn sensible behaviour. With Recency barred, importance concentrates on Tenure (0.370) and purchase cadence (0.219) (Figure 4.10): short tenure, irregular customers churn. This is actionable — it says *which* behavioral precursors the retention campaigns of Section 4.11 should watch.

Error analysis in marketing terms. The confusion counts of Table 5.4 translate directly into campaign economics (the asymmetry argued in Section 2.1.4). Random Forest's hold out errors are 237 false positives and 29 false negatives. The 237 false positives each cost, at worst, one unnecessary retention contact pennies to a few pounds under the channel costs of Section 4.12. The 29 false negatives are silently departing customers; at the median CLV of £371 (let alone the £1,418 mean), a single missed churner can outweigh dozens of wasted contacts. Logistic regression pushes the same trade further (349 FP, 13 FN; recall 0.889), which is why it remains competitive on the ranking despite the lowest precision: under this cost structure, recall heavy operating points are the economically rational ones, and the degenerate models (Gradient Boosting: 15 FP, 113 FN) are the expensive ones however clean their ROC curves look.

Robustness to the label threshold. Because the churn label is a proxy defined at the 90th Recency percentile (Section 4.9), the entire six model comparison was repeated with the threshold moved to the 85th and the 95th percentiles the label rebuilt, every model retrained, and the same stratified hold-out rescored (Table 5.7). The conclusions survive both moves.

Held-out ROC-AUC stays within 0.78-0.85 for every model at every threshold, and the top of the ranking remains the class-weighted ensembles and logistic regression, whose differences stay inside the noise band the McNemar analysis established at the 90th percentile. What does change is the operating regime: as the positive class shrinks from 15% to 5% of the base, PR-AUC falls for every model the expected arithmetic of rarer positives, not a property of any classifier. The modelling conclusions are therefore not an artefact of one labelling choice; the deployed threshold should be chosen against campaign economics, and Section 5.9 records the proxy nature of the label as a residual limitation.

Table 5.7 Sensitivity to the churn-label threshold: held-out ROC-AUC for each model when the label is redefined at the 85th, 90th, and 95th Recency percentiles (churn rates 15%, 10%, 5%; thresholds 449, 535, 625 days).

Model	P85 (15% churn)	P90 (10% churn)	P95 (5% churn)
Logistic Regression	0.834	0.846	0.839
Random Forest	0.838	0.849	0.827
XGBoost	0.828	0.847	0.832
Gradient Boosting	0.828	0.845	0.821
Decision Tree	0.825	0.828	0.813
KNN	0.779	0.808	0.794

5.3.3 Verdict against O4 (churn half)

Six classifiers trained and compared under a strict leakage guard, imbalance handling where intended, threshold-aware metrics, cross-validation, and formal significance testing: O4's churn requirement is met, with the statistical tie honestly reported rather than resolved by decree.

5.4 Evaluation of the CLV Model

The BG/NBD + Gamma Gamma fit (Section 4.8) passes its admissibility diagnostic (near zero frequency value correlation) and produces the expected long-tailed value structure: median CLV £371 against mean £1,418; the top decile of customers holds a disproportionate share of the £8.33M portfolio value. P(alive) (Eq. 12) is high for most of the base (median 0.980) with a decisive at-risk minority mean 0.909 and the expected purchase forecasts scale consistently across the 90/180/365 day horizons (means 0.80, 1.58, 3.17 purchases). Their role in the system is as *relative* value and activity rankings feeding the tiering of Section 4.11, for which the long-tail shape and stable ordering are the properties that matter, and both hold.

Temporal validation on a one-year holdout. The purchasing half of the model was additionally validated out-of-sample with the standard Fader Hardie calibration/holdout procedure [17]: the transaction history was split at 9 December 2010, BG/NBD was refitted on the first trading year alone (4,311 customers active in that window), and its forecast of each customer's purchase count over the entirely unseen second year was compared with what those customers actually did. (The shorter calibration window required care in fitting: small L2 penalties drove the dropout parameters to a degenerate boundary, so the reported fit is the cleanly converged unpenalized one; the fitting ladder and its diagnostics are implemented and logged in src/clv_validation.py.) The result validates the model for exactly the role it plays, and qualifies it for the role it does not. Per customer discrimination is strong: predicted and actual holdout purchase counts correlate at r = 0.83 (MAE 2.51 purchases against a holdout mean of 2.86), and the left panel of Figure 5.3 shows the classic conditional expectation check mean actual holdout purchases rise monotonically with calibration-period frequency exactly as the model orders them, reproducing on this dataset the qualitative result of [17]. Absolute calibration is weaker: the model over forecasts total holdout purchases by 52.9% (18,823 predicted against 12,315 actual), visible in Figure 5.3 as the predicted curve sitting uniformly above the actual (left) and every decile of the reliability panel falling below the diagonal (right); the excess traces to the fitted dropout process, which under estimates year 2 attrition. The two-sided conclusion is deliberate: the CLV layer is validated as a ranking instrument which is precisely how the tiering of Section 4.11 consumes it while its absolute currency forecasts should be read as optimistic upper bounds, a caveat now recorded in Section 5.9.

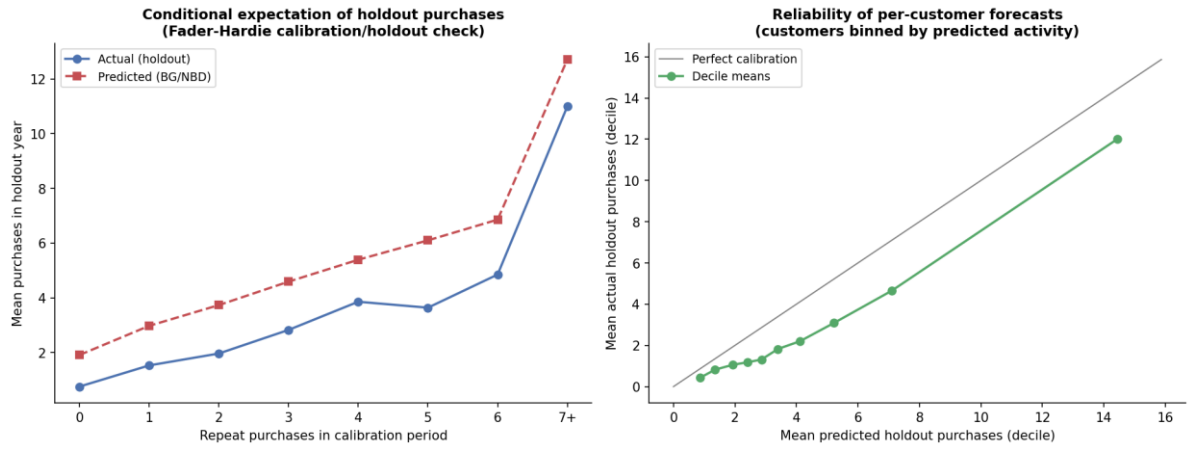

Figure 5.3 Temporal validation of the BG/NBD purchase model on a one-year holdout. Left: mean actual versus predicted holdout purchases grouped by calibration-period frequency (the Fader Hardie conditional expectation check) the ordering is reproduced, with a uniform upward bias. Right: decile level reliability of the per customer forecasts strongly monotone, consistently below the perfect calibration diagonal.

5.5 Evaluation of the Migration Analysis

For the 2,772 customers active in both trading years, Tables 5.8–5.9 and Figure 5.4 give the year-1 → year-2 transition structure. Three patterns validate that the segments describe real, persistent behavioural states rather than noise. **Champions are sticky**: 61.7% of year-1 Champions remain Champions in year 2 the strongest diagonal in the matrix. **General Customers act as a gravity well**: 45.3% remain, and every other segment leaks into it, which is exactly what a mainstream behavioural state should look like. **Loss is persistent**: 43.4% of year-1 Lost Customers remain Lost, and the At Risk segment feeds Lost at 30.0% the decay path the retention interventions of Section 4.11 exist to interrupt. That the matrix is strongly diagonal *without* the segmentation being trained on year-to-year consistency is independent evidence for the behavioural reality of the segments (O3), and the migration view itself discharges FR10.

Table 5.8 Year-1 → year-2 segment migration: raw customer counts (2,772 customers active in both years).

segment_y1	Champions	Loyal Customers	Big Spenders	Potential Loyalists	General Customers	At Risk	Lost Customers
Champions	222	37	11	8	66	3	13
Loyal Customers	40	36	4	27	69	4	16
Big Spenders	9	3	5	3	6	0	4
Potential Loyalists	18	20	3	316	237	12	230
General Customers	48	50	4	213	443	23	197
At Risk	1	0	1	12	14	0	12
Lost Customers	5	4	0	105	73	1	144

Table 5.9 Year-1 → year-2 segment migration: row-normalised transition rates.

segment_y1	Champions	Loyal Customers	Big Spenders	Potential Loyalists	General Customers	At Risk	Lost Customers
Champions	0.6167	0.1028	0.0306	0.0222	0.1833	0.0083	0.0361
Loyal Customers	0.2041	0.1837	0.0204	0.1378	0.352	0.0204	0.0816
Big Spenders	0.3	0.1	0.1667	0.1	0.2	0	0.1333
Potential Loyalists	0.0215	0.0239	0.0036	0.378	0.2835	0.0144	0.2751
General Customers	0.0491	0.0511	0.0041	0.2178	0.453	0.0235	0.2014
At Risk	0.025	0	0.025	0.3	0.35	0	0.3
Lost Customers	0.0151	0.012	0	0.3163	0.2199	0.003	0.4337

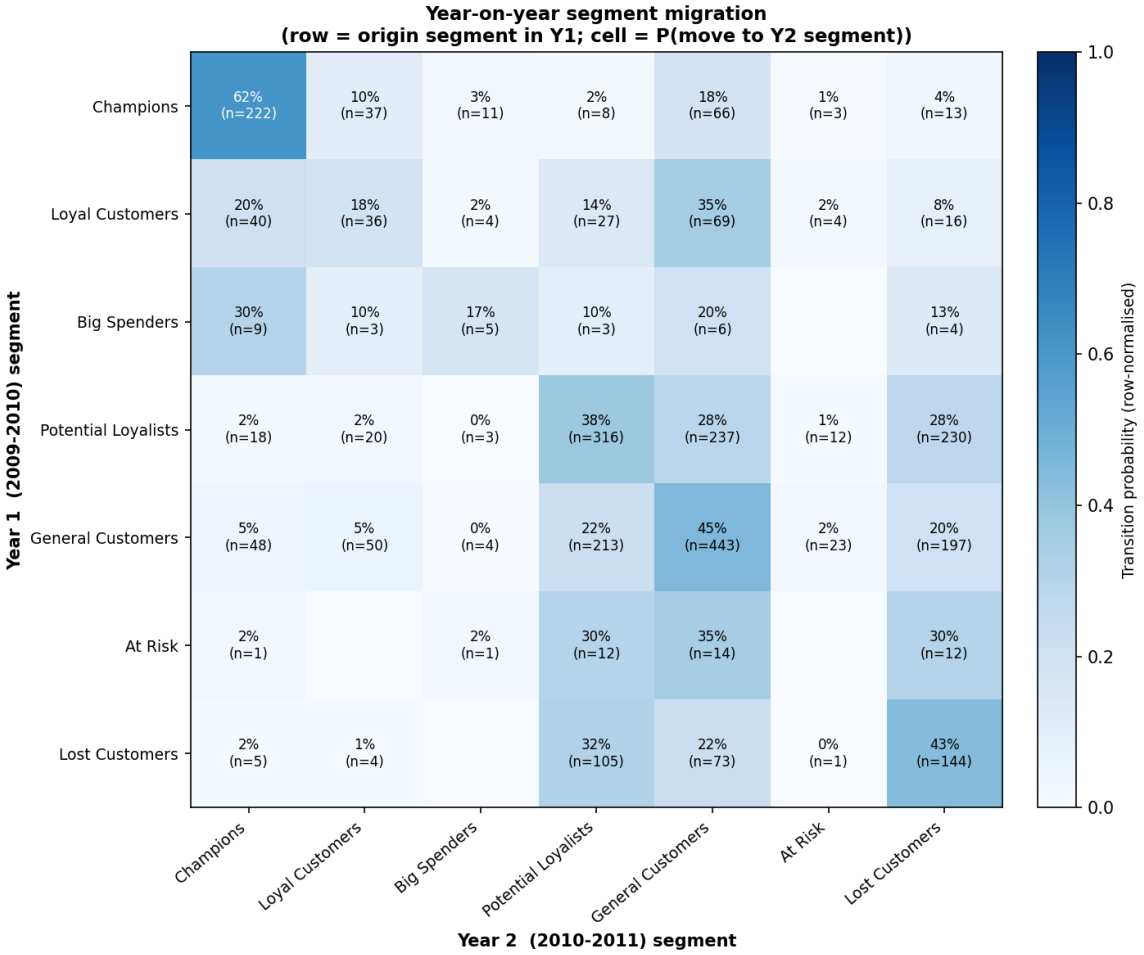

Figure 5.4 Segment migration heatmap between the two trading years.

5.6 Evaluation of the Notification Plan

The engine plans a campaign for every one of the 5,878 customers, and the *shape* of the plan is its evaluation: it must concentrate expensive attention narrowly and cheap attention broadly. It does. The action distribution runs from 3,023 low cost engagement promotions down to exactly **64 priority retention interventions** (1.1% of the base) the only action carrying the expensive channel mix (Email + SMS + personal outreach; the remaining 5,814 customers are contacted by email alone). Priorities stack sensibly: 443 customers at priority 1, 2,413 at 2, 1,400 at 3, 1,620 at 4, and only 2 at the capped maximum of 5. The value and risk tiers driving these decisions split the base into 1,176 High / 2,351 Medium / 2,351 Low value, and 1,717 High / 707 Medium / 3,454 Low churn risk. Cross checking the engine's logic end to end (also covered by five unit tests, Section 5.1): the 64 intervention recipients are exactly the High/Medium value customers in the High churn band after segment-playbook modulation the auditable trace that NFR5 demands. On demand lookups through `recommend(customer_id)` return the identical decision for any single customer, satisfying FR11's API requirement and powering the dashboard lookup page (Figure 4.11 ff.). The

dashboard delivery layer has so far been assessed technically it renders the persisted artefacts faithfully and its live lookup agrees with the engine's batch output; a structured protocol for expert heuristic evaluation against Nielsen's usability heuristics [43] is provided in Appendix C, with a practitioner study noted as future work (Section 6.2).

5.7 Evaluation of the Commercial ROI

Table 5.10 and Figure 5.5 report the Monte Carlo results (10,000 iterations). The targeted plan costs **£623.50** to contact all 5,878 customers and yields a mean simulated profit of **£23,952** a mean ROI of **38.4×** (median 37.4×), with a 95% credible interval of **[21.8×, 60.9×]** and a positive return in 100% of simulated worlds. The static blanket baseline the same 5,878 customers, one generic email, 2% response costs £293.90 and yields £8,927 mean profit (ROI 30.4×). The targeted plan therefore delivers a **profit uplift of +£15,025, or +168.3%**, for roughly double the (tiny) contact cost. Three properties make this a defensible business case rather than a cherry picked number: the result is a full distribution, not a point estimate; the headline is the *uplift*, which is robust to the shared planning assumptions in a way the absolute ROI is not (both plans use the same margin, noise, and cost model, so assumption errors largely cancel in the difference); and every input assumption response priors from 2% to 30%, margin 30%, discount 10%, per-channel costs is documented in one configuration module and can be varied by a sceptical reader. This discharges O6 and FR12.

Table 5.10 Monte Carlo ROI summary (10,000 iterations), including the static-baseline comparison and uplift (Eqs. 24–27).

	value
n_simulations	10,000
total_cost	623.5
mean_gross_revenue	1.229e+05
mean_contribution	2.458e+04
mean_profit	2.395e+04
mean_roi	38.42
median_roi	37.35
roi_ci_low_95	21.82
roi_ci_high_95	60.93
prob_positive_roi	1
prob_roi_gt_1	1
static_total_cost	293.9
static_mean_profit	8,927
static_mean_roi	30.38
roi_uplift_vs_static	8.041

	value
profit_uplift_vs_static	1.503e+04
profit_uplift_pct_vs_static	168.3

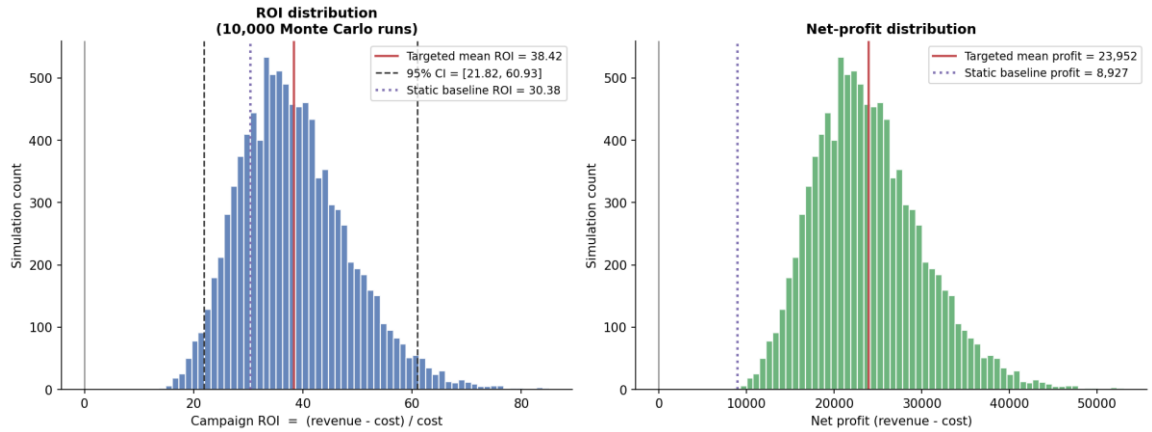

Figure 5.5 Simulated ROI (left) and net-profit (right) distributions for the targeted plan, with the static blanket baseline overlaid.

Sensitivity of the uplift to the planning assumptions. The business case rests on documented assumptions, so each was deliberately stressed and the full 10,000-run simulation repeated for both plans under every perturbation (Table 5.11, Figure 5.6). The uplift survives every single-assumption attack. Halving all targeted response priors while leaving the blanket baseline untouched the most conservative single move available still leaves a positive mean uplift (+£2,739; P(uplift > 0) = 0.69). Doubling the baseline's response rate to 4%, doubling or even quintupling every channel cost, cutting gross margin to 20%, and deepening the offer discount to 15% leave the mean uplift between +£5,889 and +£14,695 with P(uplift > 0) ≥ 0.74. Only the compound worst case all five pessimistic moves imposed simultaneously turns the mean uplift negative (−£2,176), and even there the targeted plan itself remains profitable in absolute terms (mean ROI 1.5×; P(ROI > 0) = 0.99). The honest reading is therefore not that targeting is unconditionally superior, but that reversing the conclusion requires every planning assumption to be wrong, in the plan's disfavour, at the same time. The absolute ROI figures scale with the assumptions; the sign of the uplift is what survives them which is exactly why Section 4.12 designed the evaluation around the difference between two identically simulated plans.

Table 5.11 ROI sensitivity: mean ROI of both plans, mean profit uplift, and P(uplift > 0) under each perturbed-assumption scenario (10,000 Monte Carlo runs per scenario).

Scenario	**Targeted mean ROI**	**Static mean ROI**	**Mean profit uplift (£)**	**P(uplift > 0)**
Baseline (as reported)	38.4×	30.4×	+15,025	0.962

Scenario	Targeted mean ROI	Static mean ROI	Mean profit uplift (£)	P(uplift > 0)
Response priors ×0.5	18.7×	30.4×	+2,739	0.694
Response priors ×0.75	28.5×	30.4×	+8,809	0.884
Response priors ×1.5	58.2×	30.4×	+27,357	0.995
Static response ×2 (4%)	38.4×	61.5×	+5,889	0.738
Channel costs ×2	18.7×	14.7×	+14,695	0.960
Channel costs ×5	6.9×	5.3×	+13,707	0.949
Gross margin 30% → 20%	18.7×	14.7×	+7,348	0.960
Offer discount 10% → 15%	28.6×	22.5×	+11,186	0.961
Worst case (all pessimistic)	1.5×	6.8×	−2,176	0.046

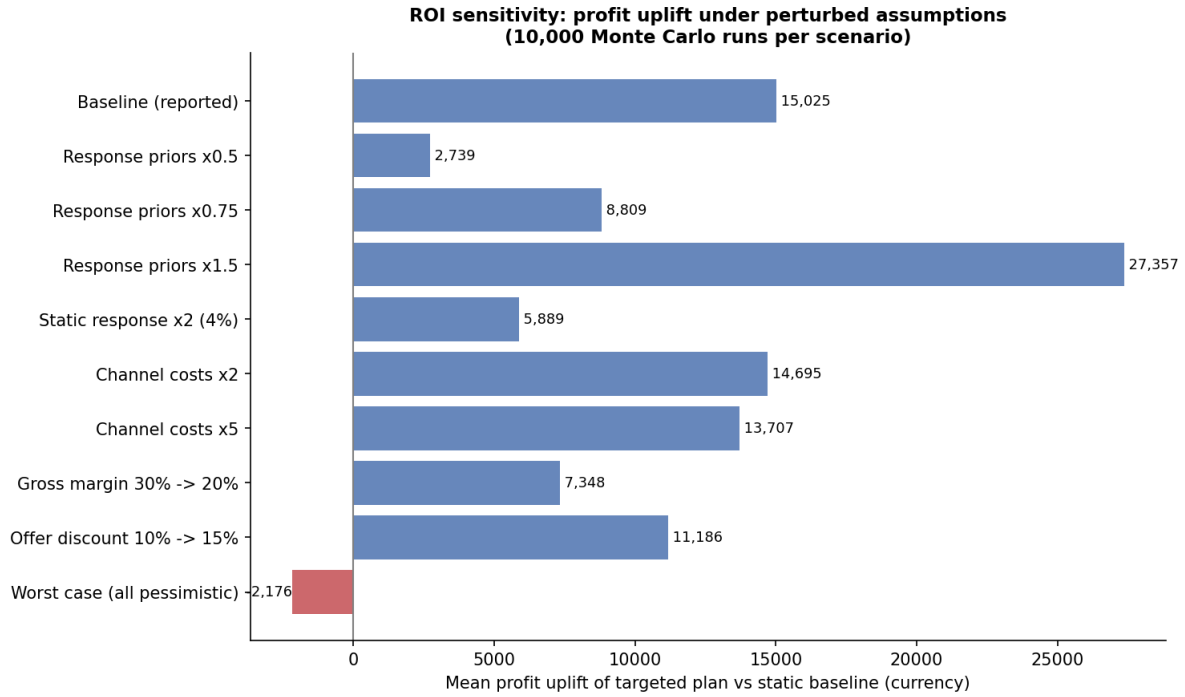

Figure 5.6 Mean profit uplift of the targeted plan over the static baseline under each perturbed assumption scenario. The uplift survives every single assumption stress; only the compound worst case all assumptions moved against the plan at once turns it negative.

5.8 Requirements Traceability

Every functional requirement is implemented and evidenced:

Requirement	Implemented in	Evidence
FR1 Ingestion	src/data_loading.py	1,067,371 rows loaded (Section 4.2)
FR2 Auditable cleaning	src/cleaning.py	Table 4.1; tests (Section 5.1)
FR3 Features	src/features.py	Section 4.3, Figure 4.1
FR4 Preprocessing	src/preprocessing.py	Figure 4.2; tests
FR5 Six-algorithm clustering	src/clustering.py	Section 4.5, Figures 4.3–4.6
FR6 Validation & selection	src/validation.py	Tables 5.2–5.3, Figure 5.1
FR7 Profiling & naming	src/profiling.py	Table 4.4, Figures 4.7–4.8
FR8 CLV	src/clv.py	Table 4.5, Figure 4.9, Section 5.4
FR9 Churn	src/churn.py	Tables 5.4–5.6, Figures 4.10, 5.2
FR10 Migration	src/migration.py	Tables 5.8–5.9, Figure 5.4
FR11 Notification plan + API	src/notifications.py	Table 4.6, Section 5.6
FR12 ROI + uplift	src/roi.py	Table 5.10, Figure 5.5
FR13 Dashboard	app/streamlit_app.py	Figures 4.11–4.13

The non-functional requirements are evidenced as follows: NFR1 by byte identical repeated runs (Section 5.1); NFR2/NFR3 by the architecture of Chapter 3 as implemented; NFR4 by the ≈160 s CPU runtime; NFR5 by the rule-traceable plan audit of Section 5.6; NFR6 by the 20-test suite.

5.9 Threats to Validity

Honest limits qualify the results. Scale and coarseness: the customer base (5,878) is modest and the selected segmentation is coarse (two clusters plus noise); a larger or more heterogeneous base might support and reward finer structure. Single dataset: all findings derive from one UK online retailer over 2009–2011; the pipeline is dataset-agnostic by construction, but the conclusions (e.g. HDBSCAN's win) are not claimed to generalise beyond this data, consistent with the no-free-lunch finding of the comparative literature [20], [21], [40]. CLV calibration: the temporal holdout of Section 5.4 validates the CLV layer as a ranking instrument ($r = 0.83$ against unseen year-2 behaviour) but shows its absolute purchase forecasts run ≈53% high on this data, so portfolio-value figures should be read as optimistic upper bounds. Churn label is a proxy: the label marks the most dormant decile rather than observed contract termination an unavoidable construction in non-contractual retail though Section 5.3.2 shows the modelling conclusions are stable when the defining percentile is moved to 85 or 95. ROI priors are planning assumptions: the response rates are documented assumptions, not fitted from a live campaign; this is why the analysis emphasises the uplift over the baseline (Eq. 27) rather than the absolute ROI, why Section

5.7 stresses every assumption individually (the uplift survives each one; only the compound worst case reverses it), and why future work proposes A/B calibrated uplift modelling. Stationarity: churn and CLV both assume historical behaviour patterns persist; seasonal shifts or market shocks would require re fitting cheap at 160 s per run, but a monitoring obligation in any deployment. Interpretability trade off: the notification engine keys on cluster derived segment names rather than raw cluster indices, trading a little cluster fidelity for a decision table a marketer can read a deliberate choice aligned with NFR5.

Chapter 6
Conclusions and Future Work

6.1 Conclusions

This project set out to close the gap, identified across the segmentation, CLV, and churn literatures, between analytical modelling and marketing action. It delivered a reproducible software framework that carries raw retail transactions end to end: 1,067,371 rows cleaned to 793,591 under four audited rules; seven behavioural features engineered for 5,878 customers; six clustering algorithms benchmarked under three validity indices and 50-round bootstrap stability, with HDBSCAN selected by a transparent, pre declared rule (silhouette 0.416, best overall rank 1.75); probabilistic 12-month CLV (portfolio £8.33M, median £371), temporally validated as a ranking instrument on a one-year holdout (r = 0.83) and leakage-guarded churn prediction (best ROC-AUC 0.849) with McNemar testing showing the two best models statistically tied; a rule-based engine translating segment, value tier, and risk band into a prioritised campaign per customer 64 priority retention interventions among 5,878 planned contacts and a 10,000 run Monte Carlo assessment showing a mean ROI of 38.4× and a +168% profit uplift over blanket marketing an uplift that survives every single assumption stress test; all surfaced through an eight-page dashboard with live per-customer lookup.

All six objectives are discharged: O1 by the survey and method justification of Chapter 2; O2 by the audited data foundation of Sections 4.2-4.4; O3 by the validated, rule selected segmentation (Sections 4.5-4.7, 5.2); O4 by the CLV and churn layers (Sections 4.8-4.9, 5.3-5.4); O5 by the notification engine, API, and dashboard (Sections 4.11, 4.13, 5.6); and O6 by the uplift-quantified business case (Sections 4.12, 5.7).

Two conclusions rise above the implementation. First, *rigour changes answers*: the evidence-based selection rejected the conventional default (K-Means) in favour of HDBSCAN, the significance test blocked an unjustified "best model" claim, and the imbalance-handling contrast exposed two models whose respectable ROC-AUC concealed operational uselessness none of which a single-algorithm, single-metric study would have surfaced. Second, *the closed loop is the contribution*: validated clusters, value forecasts, and risk scores only become worth their computation when a traceable decision layer converts them into actions whose financial consequence is quantified before spend and this project demonstrates that the whole loop fits in one auditable, three-minute, commodity hardware pipeline.

6.2 Future Work

Five extensions follow naturally, in rough order of value:

- **Campaign-calibrated uplift modelling.** Replace the documented response priors with rates estimated from live A/B tests, and ultimately model per-customer *treatment effects* with causal uplift methods [38], [39] turning the indicative ROI into a measured one and targeting customers by *incremental* response rather than risk alone.
- **Deeper temporal validation of the predictive layer.** Section 5.4 now validates the BG/NBD purchase forecasts on a one-year holdout; the natural extensions are validating the monetary half (Gamma–Gamma) and the churn probabilities against realised future behaviour, and diagnosing the dropout under-estimation behind the ≈53% aggregate over-forecast — for example with rolling-origin splits or a Pareto/NBD comparison.
- **Real-time deployment.** Wrap `recommend(customer_id)` behind a service API fed by streaming transactions, recomputing features incrementally so recommendations update as behaviour changes realising the dynamic-CRM vision of [24], [36] that the batch pipeline approximates.
- **Richer representations at larger scale.** On a larger customer base, explore learned behavioural embeddings (e.g. autoencoders over transaction sequences) as clustering inputs, and GPU accelerated implementations (RAPIDS cuML) if scale demands it; the validation framework of Section 4.6 transfers unchanged, which is the point of having built it.
- **Human-subject evaluation of the dashboard.** A structured usability study with marketing practitioners would evaluate the delivery layer as seriously as this report evaluates the analytical layers, and would likely refine the playbook's actions and offers. The heuristic-evaluation protocol of Appendix C is the prepared starting point.

List of References

[1] J. MacQueen, "Some methods for classification and analysis of multivariate observations," in *Proc. 5th Berkeley Symp. Math. Statist. Probab.*, Berkeley, CA, 1967, pp. 281–297.

[2] S. P. Lloyd, "Least squares quantization in PCM," *IEEE Trans. Inf. Theory*, vol. 28, no. 2, pp. 129–137, 1982.

[3] M. Ester, H.-P. Kriegel, J. Sander, and X. Xu, "A density-based algorithm for discovering clusters in large spatial databases with noise," in *Proc. 2nd Int. Conf. Knowledge Discovery and Data Mining (KDD)*, 1996, pp. 226–231.

[4] E. Schubert, J. Sander, M. Ester, H.-P. Kriegel, and X. Xu, "DBSCAN revisited, revisited: Why and how you should (still) use DBSCAN," *ACM Trans. Database Syst.*, vol. 42, no. 3, pp. 1–21, 2017.

[5] R. J. G. B. Campello, D. Moulavi, and J. Sander, "Density-based clustering based on hierarchical density estimates," in *Proc. Pacific-Asia Conf. Knowledge Discovery and Data Mining (PAKDD)*, 2013, pp. 160–172.

[6] L. McInnes, J. Healy, and S. Astels, "hdbscan: Hierarchical density based clustering," *J. Open Source Softw.*, vol. 2, no. 11, p. 205, 2017.

[7] A. P. Dempster, N. M. Laird, and D. B. Rubin, "Maximum likelihood from incomplete data via the EM algorithm," *J. Roy. Statist. Soc. B*, vol. 39, no. 1, pp. 1–38, 1977.

[8] L. Breiman, "Random forests," *Mach. Learn.*, vol. 45, no. 1, pp. 5–32, 2001.

[9] T. Chen and C. Guestrin, "XGBoost: A scalable tree boosting system," in *Proc. 22nd ACM SIGKDD Int. Conf. Knowledge Discovery and Data Mining*, 2016, pp. 785–794.

[10] V. Satopää, J. Albrecht, D. Irwin, and B. Raghavan, "Finding a 'kneedle' in a haystack: Detecting knee points in system behavior," in *Proc. 31st Int. Conf. Distributed Computing Systems Workshops*, 2011, pp. 166–171.

[11] P. J. Rousseeuw, "Silhouettes: A graphical aid to the interpretation and validation of cluster analysis," *J. Comput. Appl. Math.*, vol. 20, pp. 53–65, 1987.

[12] D. L. Davies and D. W. Bouldin, "A cluster separation measure," IEEE Trans. Pattern Anal. Mach. Intell., vol. PAMI-1, no. 2, pp. 224–227, 1979.

[13] T. Caliński and J. Harabasz, "A dendrite method for cluster analysis," *Commun. Statist.*, vol. 3, no. 1, pp. 1–27, 1974.

[14] L. Hubert and P. Arabie, "Comparing partitions," *J. Classification*, vol. 2, no. 1, pp. 193–218, 1985.

[15] D. Chen, S. L. Sain, and K. Guo, "Data mining for the online retail industry: A case study of RFM model-based customer segmentation using data mining," *J. Database Marketing & Customer Strategy Manag.*, vol. 19, no. 3, pp. 197–208, 2012.

[16] P. S. Fader, B. G. S. Hardie, and K. L. Lee, "RFM and CLV: Using iso-value curves for customer base analysis," *J. Marketing Res.*, vol. 42, no. 4, pp. 415–430, 2005.

[17] P. S. Fader, B. G. S. Hardie, and K. L. Lee, "'Counting your customers' the easy way: An alternative to the Pareto/NBD model," *Marketing Sci.*, vol. 24, no. 2, pp. 275–284, 2005.

[18] Q. McNemar, "Note on the sampling error of the difference between correlated proportions or percentages," *Psychometrika*, vol. 12, no. 2, pp. 153–157, 1947.

[19] F. Pedregosa et al., "Scikit-learn: Machine learning in Python," *J. Mach. Learn. Res.*, vol. 12, pp. 2825–2830, 2011.

[20] J. M. John, O. Shobayo, and B. Ogunleye, "An exploration of clustering algorithms for customer segmentation in the UK retail market," arXiv:2402.04103, 2024.

[21] A. S. Paramita and T. Hariguna, "Comparison of K-Means and DBSCAN algorithms for customer segmentation in e-commerce," J. Digit. Market. Digit. Currency, vol. 1, no. 1, pp. 43–62, 2024.

[22] S. Ibrahim, B. S. Tawfik, M. A. Makhlouf, and B. Hafiz, "A novel hybrid deep learning framework for customer churn prediction using RFM and embedding clustering," Sci. Rep., vol. 16, Art. no. 16563, 2026.

[23] P. Chapman et al., *CRISP-DM 1.0: Step-by-Step Data Mining Guide*. SPSS Inc., 2000.

[24] S. Dash, "Leveraging machine learning algorithms in enterprise CRM architectures for personalized marketing automation," J. Artif. Intell. Res. (The Science Brigade), vol. 4, no. 1, pp. 482–518, 2024.

[25] A. J. Christy, A. Umamakeswari, L. Priyatharsini, and A. Neyaa, "RFM ranking — An effective approach to customer segmentation," J. King Saud Univ. — Comput. Inf. Sci., vol. 33, no. 10, pp. 1251–1257, 2021.

[26] Y. Zhang, E. T. Bradlow, and D. S. Small, "Predicting customer value using clumpiness: From RFM to RFMC," Marketing Sci., vol. 34, no. 2, pp. 195–208, 2015.

[27] B. G. Vo, H. D. S. Van, N. D. Van, and H. D. Huynh, "Customer segmentation: Automatic K-optimization and RFM-based K-means clustering," in Proc. 10th Int. Conf.

Intelligent Information Technology (ICIIT), Hanoi, Vietnam, 2025, doi: 10.1145/3731763.3731805.

[28] N. Kumar, "Intelligent customer segmentation: Unveiling consumer patterns with machine learning," J. Umm Al-Qura Univ. Eng. Archit., vol. 16, pp. 774–783, 2025.

[29] S. Mousaeirad, "Intelligent vector-based customer segmentation in the banking industry," arXiv:2012.11876, 2020.

[30] A. Mammadzada, E. Alasgarov, and A. Mammadov, "Application of BG/NBD and Gamma-Gamma models to predict customer lifetime value for financial institution," in Proc. IEEE 15th Int. Conf. Application of Information and Communication Technologies (AICT), 2021, doi: 10.1109/AICT52784.2021.9620535.

[31] A. Wong, A. V. Garcia, and Y. Lim, "A data-driven approach to customer lifetime value prediction using probability and machine learning models," Decis. Anal. J., vol. 16, Art. no. 100601, 2025.

[32] A. Sikri, R. Jameel, S. M. Idrees, and H. Kaur, "Enhancing customer retention in telecom industry with machine learning driven churn prediction," Sci. Rep., vol. 14, Art. no. 13097, 2024.

[33] M. Shahabikargar, A. Beheshti, X. Zhang, J. Foo, and A. Jolfaei, "A comprehensive survey on customer churn analysis studies," J. Inf. Telecommun., vol. 10, no. 1, pp. 24–70, 2026.

[34] N. Jha, D. Parekh, M. Mouhoub, and V. Makkar, "Customer segmentation and churn prediction in online retail," in Advances in Artificial Intelligence (Canadian AI 2020), LNAI vol. 12109, Springer, 2020, pp. 328–334.

[35] A. Ashraf, C. A. Rayed, N. A. Awad, and H. M. Sabry, “A framework for customer segmentation to improve marketing strategies using machine learning,” Procedia Comput. Sci., vol. 260, pp. 616–625, 2025, doi: 10.1016/j.procs.2025.03.240.

[36] Y. Gaidhani, J. V. N. Ramesh, S. Singh, R. Dagar, T. S. M. Rao, S. R. Godla, and Y. A. B. El-Ebiary, "AI-driven predictive analytics for CRM to enhance retention, personalization and decision-making," Int. J. Adv. Comput. Sci. Appl., vol. 16, no. 4, 2025.

[37] L. Liu, "e-Commerce personalized recommendation based on machine learning technology," Mobile Inf. Syst., vol. 2022, Art. no. 1761579, 2022.

[38] J. Wang, Y. Tan, B. Jiang, B. Wu, and W. Liu, "Dynamic marketing uplift modeling: A symmetry-preserving framework integrating causal forests with deep reinforcement learning for personalized intervention strategies," Symmetry, vol. 17, no. 4, Art. no. 610, 2025.

[39] N. Sawant, C. B. Namballa, N. Sadagopan, and H. Nassif, "Contextual multi-armed bandits for causal marketing," arXiv:1810.01859, 2018.

[40] A. T. Thuse, "Customer segmentation for targeted marketing: Exploring DBSCAN and K-means," Preprints.org, manuscript 202504.1434, 2025.

[41] D. Chen, "Online Retail II," UCI Machine Learning Repository, 2019. [Online]. Available: https://archive.ics.uci.edu/dataset/502/online+retail+ii

[42] M. Wedel and W. A. Kamakura, *Market Segmentation: Conceptual and Methodological Foundations*, 2nd ed. Boston, MA: Kluwer Academic Publishers, 2000.

[43] J. Nielsen, "Enhancing the explanatory power of usability heuristics," in Proc. SIGCHI Conf. Human Factors in Computing Systems (CHI ’94), Boston, MA, USA, 1994, pp. 152–158.

Appendix A
External Materials

A.1 Source Code and Repository Layout

The complete source code is submitted alongside this report (and maintained under Git version control). The repository layout mirrors the architecture of Chapter 3:

```
final-year-project/
|-- main.py                  # pipeline orchestrator (run: python main.py)
|-- src/
|   |-- config.py            # ALL paths, thresholds, hyper-parameters, seeds
|   |-- data_loading.py      # Stage 1a: load both yearly worksheets
|   |-- cleaning.py          # Stage 1b: four audited cleaning rules
|   |-- features.py          # Stage 2 : RFM + extended features
|   |-- preprocessing.py     # Stage 2b: skew-gated log1p + StandardScaler
|   |-- clustering.py        # Stage 3 : six algorithms, auto hyper-parameters
|   |-- validation.py        # Stage 3b: indices, bootstrap ARI, ranking, selection
|   |-- profiling.py         # Stage 4 : profiles, naming, heatmap, radar
|   |-- clv.py               # Stage 5 : BG/NBD + Gamma-Gamma CLV
|   |-- churn.py             # Stage 6 : six classifiers, McNemar, ranking
|   |-- migration.py         # Stage 7 : year-on-year transitions
|   |-- notifications.py     # Stage 8: rule engine + recommend(customer_id)
|   |-- roi.py               # Stage 9: Monte Carlo ROI + static baseline
|   |-- clv_validation.py    # Stage 9b: BG/NBD calibration/holdout validation
|   |-- churn_sensitivity.py # Stage 9c: churn label-threshold sensitivity
|   |-- roi_sensitivity.py   # Stage 9d: ROI assumption-sensitivity scenarios
|   |-- app/streamlit_app.py     # Stage 10: eight-page dashboard
|-- scripts/                 # report generators + run_new_validations.py
|-- tests/                   # 20 pytest unit tests (7 modules)
|-- data/raw/                # source workbook (read-only)
|-- data/processed/          # 5 Parquet datasets + fitted scaler
|-- outputs/tables/          # 18 result tables (CSV)
|-- outputs/figures/         # 22 figures (PNG)
```

A.2 How to Run

With Python 3.11 and the dependencies of Section A.3 installed:

```
# full pipeline (regenerates every table and figure; ~160 s on CPU)
python main.py

# unit tests
python -m pytest tests/ -q          # expected: 20 passed


# post-draft validation analyses (Sections 5.3.2, 5.4, 5.7)
python scripts/run_new_validations.py


# dashboard (then open http://localhost:8501)
python -m streamlit run app/streamlit_app.py
```

Reproducibility: all stochastic steps read the seed (42) and the pinned snapshot date from `src/config.py`; repeated runs produce byte-identical outputs.

A.3 Software Dependencies

Python 3.11; pandas; NumPy; scikit-learn [19]; hdbscan [6]; lifetimes; XGBoost [9]; SciPy; Matplotlib; Streamlit; pytest; pyarrow (Parquet I/O); openpyxl (Excel ingestion).

A.4 Dataset

UCI Online Retail II [41]: transactions of a UK-based online giftware retailer, 1 December 2009 – 9 December 2011, across two worksheets (one per trading year); 1,067,371 rows raw. Licence: CC BY 4.0. The raw workbook is stored read-only under `data/raw/` and is never modified by the pipeline.

A.5 Artefact Inventory

Every artefact regenerated by `python main.py`, in production order:

Artefact	Kind	Produced by
cleaned_transactions.parquet	dataset	Stage 1
cleaning_summary.csv	table (Table 4.1)	Stage 1
customer_features.parquet	dataset	Stage 2
feature_summary.csv	table (Table 4.2)	Stage 2
rfm_distributions.png	figure (Fig. 4.1)	Stage 2
scaled_features.parquet; fitted_scaler.joblib	dataset + model	Stage 2b
scaling_effect.png	figure (Fig. 4.2)	Stage 2b
clustered_customers.parquet	dataset (6 label columns)	Stage 3
kmeans_metrics.csv; gmm_bic.csv	tables (Table 4.3; GMM BIC curve data)	Stage 3
kmeans_selection.png; gmm_bic.png; dbscan_kdistance.png; cluster_pca_projection.png	figures (Figs. 4.3–4.6)	Stage 3
cluster_validation.csv; cluster_ranking.csv	tables (Tables 5.2–5.3)	Stage 3b
stability_ari.png	figure (Fig. 5.1)	Stage 3b
segment_profiles.csv	table (Table 4.4)	Stage 4
segment_profiles.png; radar_profiles.png	figures (Figs. 4.7–4.8)	Stage 4
customer_clv.parquet; clv_summary.csv	dataset + table (Table 4.5)	Stage 5
clv_distribution.png	figure (Fig. 4.9)	Stage 5
customer_churn.parquet	dataset	Stage 6
churn_model_comparison.csv; churn_model_ranking.csv; churn_mcnemar_pvalues.csv	tables (Tables 5.4–5.6)	Stage 6

Artefact	Kind	Produced by
churn_roc_curves.png; churn_pr_curves.png; churn_feature_importance.png	figures (Figs. 5.2a–b, 4.10)	Stage 6
segment_migration_counts.csv; segment_migration_rates.csv	tables (Tables 5.8–5.9)	Stage 7
segment_migration.png	figure (Fig. 5.4)	Stage 7
notification_plan.csv	table (Table 4.6)	Stage 8
roi_simulation_summary.csv	table (Table 5.10)	Stage 9
roi_distribution.png	figure (Fig. 5.5)	Stage 9
system_architecture.png; component_diagram.png	figures (Figs. 3.1–3.2)	design documentation
dashboard_overview.png; dashboard_segments.png; dashboard_roi.png	figures (Figs. 4.11–4.13)	Stage 10 (captured)
clv_holdout_metrics.csv	table (Section 5.4)	Validation script
clv_holdout_validation.png	figure (Figure 5.3)	Validation script
churn_threshold_sensitivity.csv	table (Table 5.7)	Validation script
roi_sensitivity.csv	table (Table 5.11)	Validation script
roi_sensitivity.png	figure (Figure 5.6)	Validation script

Appendix B
Ethical Issues Addressed

B.1 Data Protection and Legal Compliance

The project processes only the public, secondary Online Retail II dataset [41], in which customers are pseudonymised at source as numeric IDs; no names, postal or email addresses, phone numbers, or payment instruments appear anywhere in the data. No linkage to any other dataset is performed and no re-identification of any individual is attempted. Processing follows the principles of the UK GDPR even though the data is outside its scope in practice: purpose limitation (the data is used solely for the analyses of this report), data minimisation (only the fields needed for behavioural modelling are read), and storage discipline (the raw file is read-only; all derivatives are regenerable artefacts).

B.2 Human Participants

The project involves no human participants, no surveys, no interviews, and no collection of new personal data. Accordingly, no ethical approval or participant consent was required. This was confirmed against the School's ethics guidance for computing projects at the scoping stage.

B.3 Responsible Use of Targeting Systems

Behaviour based targeting is dual-use: the same machinery that reduces irrelevant contact can, misused, enable manipulative or exploitative marketing. Three design decisions mitigate this risk structurally. First, the decision layer is rule based and fully auditable (NFR5): every recommendation traces to explicit, human-readable rules and thresholds that an operator can inspect, question, and override there is no opaque optimiser between analysis and action. Second, contact timing derives from each customer's own purchase rhythm (Eq. 29), which suppresses over-contact by construction rather than by policy promise. Third, the segmentation deliberately retains a noise category whose playbook entry is the *least* aggressive action (standard newsletter, priority 1): customers the model does not understand are contacted least, not most. Residual risks notably the use of escalating discounts for at risk customers, which could in principle train price-sensitivity are noted as matters for the deploying organisation's marketing governance rather than properties of the software.

B.4 Academic Integrity and Assistance

All third-party methods, models, and datasets are cited in the List of References. In accordance with the University’s policy on the responsible use of generative artificial

intelligence, the following assistance is declared: a generative AI assistant (Anthropic Claude, used through the Claude Code development environment) assisted with software development including implementation of the validation analyses of Sections 5.3.2, 5.4, and 5.7 with drafting and revising sections of this report, and with locating and verifying the bibliographic details of the works cited. All analytical results reported here were produced by executing the submitted pipeline and validation scripts on the dataset; the project's design decisions, interpretations, and conclusions are the author's own, and the author has reviewed and takes full responsibility for every part of the submitted work.

Appendix C
Dashboard Heuristic-Evaluation Protocol

C.1 Purpose and Status

This appendix records the structured protocol prepared for expert (heuristic) evaluation of the eight-page dashboard of Section 4.13. The protocol itself is a deliverable of this project; executing it with marketing practitioners is identified as future work (Section 6.2). It is included so that the delivery layer's assessment criteria are explicit and the evaluation is repeatable by any assessor with the running dashboard.

C.2 Method

Three to five evaluators independently walk through the dashboard while completing four representative tasks: (1) identify the largest customer segment and its share of the base; (2) list the ten highest CLV customers currently in the High churn-risk band; (3) retrieve the recommendation for a given Customer ID and trace, from the on screen information alone, why that action was chosen; and (4) report the campaign's 95% ROI credible interval and its uplift over blanket marketing. Each evaluator then rates every violation found against Nielsen's ten usability heuristics [43] on the standard 0-4 severity scale (0 = not a problem, 4 = usability catastrophe). Individual findings are consolidated into a single ranked defect list; any heuristic with a median severity ≥ 3 blocks a hypothetical release.

C.3 Heuristic Checklist

Nielsen heuristic [43]	Dashboard-specific prompt for the evaluator
Visibility of system status	Is it always clear which pipeline artefacts (and from which run) the current page reflects?

Nielsen heuristic [43]	Dashboard-specific prompt for the evaluator
Match with the real world	Are segment names, value tiers, and risk bands expressed in marketing vocabulary rather than model jargon?
User control and freedom	Can the user leave any drill-down (customer lookup, segment filter) without losing context?
Consistency and standards	Do all eight pages use the same metric names, units (£), and colour encodings as this report?
Error prevention	Does the lookup page handle unknown or malformed Customer IDs gracefully?
Recognition rather than recall	Are thresholds (CLV tier percentiles, churn bands) shown where they are used, not merely documented elsewhere?
Flexibility and efficiency	Can an experienced user reach any single answer (e.g. one customer's plan) in ≤ 3 interactions?
Aesthetic and minimalist design	Does any page show data that no marketing decision needs?
Help users recognise and recover from errors	If an artefact is missing (pipeline not yet run), does the page say what to run rather than failing?
Help and documentation	Do run instructions (Appendix A.2) suffice for a first-time evaluator to start the dashboard unaided?